OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 2

SELECTING AND MANIPULATING ABSTRACT DATA TYPES (ADTS) AND DATA STRUCTURES

SUGGESTED TIME: 4 PERIODS

IN THIS CHAPTER

This chapter introduces Abstract Data Types (ADTs) as mathematical models for data organisation, explores binary trees as a fundamental hierarchical structure, and shows how ADTs solve real-world problems. By the end, you will be able to select, implement, and manipulate the right ADT for any algorithmic task in the GCE Advanced Level Computer Science exam.

- Define an Abstract Data Type and distinguish it from a concrete data structure.
- Specify the core operations of common ADTs (stack, queue, list, set, map, tree).
- Describe binary tree terminology, properties, and both array-based and linked representations.
- Implement recursive and iterative traversals (pre-order, in-order, post-order, level-order).
- Apply stacks to expression evaluation and queues to scheduling problems.
- Choose the most appropriate ADT for a given problem and justify the choice.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Abstract Data Types: Foundations
2. Binary Tree Concepts and Representation
3. Applications of ADTs
4. Integration Activity & Exam Preparation
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- Define an Abstract Data Type and distinguish it from a concrete data structure.
- Specify the core operations of common ADTs (stack, queue, list, set, map, tree).
- Describe binary tree terminology, properties, and both array-based and linked representations.
- Implement recursive and iterative traversals (pre-order, in-order, post-order, level-order).
- Apply stacks to expression evaluation and queues to scheduling problems.
- Choose the most appropriate ADT for a given problem and justify the choice.
- Analyse time and space complexity of basic ADT operations.
- Design a small integrated system (e.g., library management) using multiple ADTs.

PREREQUISITES

- Basic programming constructs (variables, loops, functions) from Lower Sixth.
- Understanding of arrays, pointers/references, and recursion.
- Familiarity with algorithmic notation (pseudocode, flowcharts).
- Concept of complexity analysis (Big-O) introduced earlier.

1 ABSTRACT DATA TYPES: FOUNDATIONS

1.1 Introduction: From the Market to the Machine

Imagine a busy market in Yaoundé where vendors must quickly serve customers, keep track of stock, and prioritise urgent orders. A fruit seller stacks mangoes in a neat pyramid — the last mango placed on top is the first one sold. A tailor keeps a notebook of client measurements — she can look up any client by name, add new clients, or remove old ones. At the bus station, passengers form a queue: the person who arrives first boards first. The way these vendors organise queues, stack goods, and catalogue items mirrors the **abstract data types** that power modern software. Understanding these structures lets you design efficient solutions for everyday Cameroonian challenges — from managing a hospital's patient waiting list to optimising a mobile money transaction log.

In this section, we formalise these intuitive ideas. We will learn what an **Abstract Data Type (ADT)** is, how it differs from a concrete **data structure**, and how to specify the behaviour of the core ADTs required by the syllabus: **Stack**, **Queue**, **List**, **Set**, **Map**, and **Tree**. We will also see how **encapsulation** and **information hiding** make our programs modular and maintainable.

1.2 What Is an Abstract Data Type?

DEFINITION **Abstract Data Type (ADT)**

An Abstract Data Type (ADT) is a mathematical model that defines:

- A set of values (the data it can hold), and
- A set of operations that can be performed on those values,

without specifying how the data is stored or how the operations are implemented. An ADT describes what the data type does, not how it does it.

Think of an ADT as a **contract** or an **interface**. The client code (the part of the program that uses the ADT) knows only the operations and their expected behaviour. The implementation details — arrays, linked lists, memory layout — are hidden behind this interface.

PROPERTY **Encapsulation and Information Hiding**

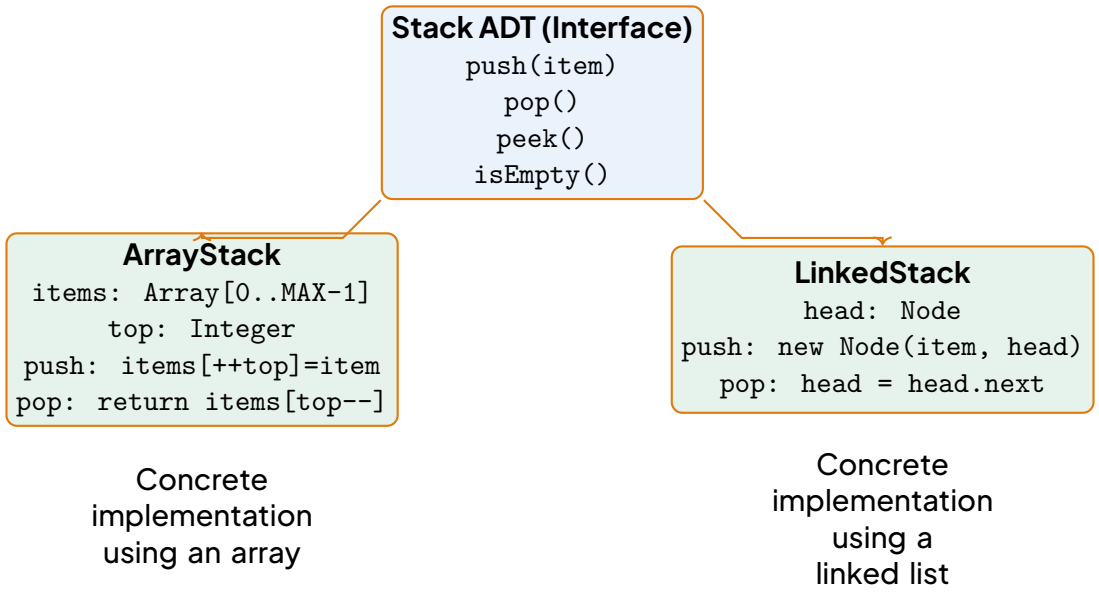
Encapsulation is the principle of bundling data and the operations that manipulate that data into a single unit (the ADT), and restricting direct access to the internal representation. Information hiding ensures that the client cannot depend on implementation details. This allows the implementation to be changed (e.g., switching from an array-based stack to a linked-list stack) without breaking any client code.

WARNING **Common Mistake: Confusing ADT with Implementation**

Many students confuse the ADT (e.g., `Stack`) with a particular implementation (e.g., `ArrayStack` or `LinkedStack`). Remember:

- Stack is an ADT: it specifies `push`, `pop`, `peek`, `isEmpty`.
- ArrayStack is a data structure: it uses an array and an integer `top` index.
- LinkedStack is another data structure: it uses a linked list with a `head` pointer.

Both `ArrayStack` and `LinkedStack` implement the `Stack` ADT. In examinations, if a question asks for the ADT, describe the operations and their semantics. If it asks for an implementation, give the concrete representation and algorithms.



An ADT (Stack) separates the interface from two possible concrete implementations.

1.3 Core ADTs Required by the Syllabus

The GCE Advanced Level syllabus expects you to know the following six fundamental ADTs. For each, we list the **essential operations**, their **parameters**, **return values**, and typical **exceptions** (pre-condition violations).

1.4 Stack ADT — Last-In, First-Out (LIFO)

A **Stack** models a pile of objects where insertion and removal happen at the same end, called the **top**. The last element pushed is the first popped.

Operation	Parameters	Return	Description / Pre-condition
`push(item)`	`item: Element`	`void`	Adds `item` to the top. Pre: stack not full (if bounded).
`pop()`	`none`	`Element`	Removes and returns the top element. Pre: `!isEmpty()`.
`peek()` or `top()`	`none`	`Element`	Returns the top element without removing it. Pre: `!isEmpty()`.
`isEmpty()`	`none`	`Boolean`	Returns `true` iff the stack contains no elements.
`size()`	`none`	`Integer`	Returns the number of elements in the stack.

1.5 Queue ADT — First-In, First-Out (FIFO)

A **Queue** models a waiting line. Elements are inserted at the **rear** (or **tail**) and removed from the **front** (or **head**).

Operation	Parameters	Return	Description / Pre-condition
`enqueue(item)`	`item: Element`	`void`	Adds `item` to the rear. Pre: queue not full (if bounded).
`dequeue()`	`none`	`Element`	Removes and returns the front element. Pre: `!isEmpty()`.
`front()` or `peek()`	`none`	`Element`	Returns the front element without removing it. Pre: `!isEmpty()`.
`isEmpty()`	`none`	`Boolean`	Returns `true` iff the queue contains no elements.
`size()`	`none`	`Integer`	Returns the number of elements in the queue.

1.6 List ADT — Ordered Collection with Positional Access

A **List** stores a sequence of elements where each element has a **position** (index). It allows insertion, deletion, and access at any position.

Operation	Parameters	Return	Description / Pre-condition
`insert(pos, item)`	`pos: Integer, item: Element`	`void`	Inserts `item` at position `pos` (0-based). Pre: 0 ≤ `pos` ≤ `size()`.
`remove(pos)`	`pos: Integer`	`Element`	Removes and returns element at `pos`. Pre: 0 ≤ `pos` < `size()`.
`get(pos)`	`pos: Integer`	`Element`	Returns element at `pos` without removing. Pre: 0 ≤ `pos` < `size()`.
`set(pos, item)`	`pos: Integer, item: Element`	`Element`	Replaces element at `pos` with `item`, returns old. Pre: 0 ≤ `pos` < `size()`.
`indexOf(item)`	`item: Element`	`Integer`	Returns first position of `item`, or -1 if not found.
`isEmpty()`	`none`	`Boolean`	Returns `true` iff list has no elements.
`size()`	`none`	`Integer`	Returns number of elements.

1.7 Set ADT — Unordered Collection of Distinct Elements

A **Set** models the mathematical notion of a set: no duplicates, no inherent order.

Operation	Parameters	Return	Description / Pre-condition
`add(item)`	`item: Element`	`Boolean`	Adds `item` if not present; returns `true` if added.
`remove(item)`	`item: Element`	`Boolean`	Removes `item` if present; returns `true` if removed.
`contains(item)`	`item: Element`	`Boolean`	Returns `true` iff `item` is in the set.
`union(other)`	`other: Set`	`Set`	Returns a new set with all elements from this and `other`.
`intersection(other)`	`other: Set`	`Set`	Returns a new set with elements common to both.
`difference(other)`	`other: Set`	`Set`	Returns a new set with elements in this but not in `other`.
`isEmpty()`	`none`	`Boolean`	Returns `true` iff set has no elements.
`size()`	`none`	`Integer`	Returns number of elements.

1.8 Map ADT — Key-Value Associations (Dictionary)

A **Map** (also called **Dictionary** or **Associative Array**) stores pairs (`key`, `value`). Keys are unique; each key maps to exactly one value.

Operation	Parameters	Return	Description / Pre-condition
`put(key, value)`	`key: Key, value: Value`	`Value`	Associates `value` with `key`. Returns previous value or `null`.
`get(key)`	`key: Key`	`Value`	Returns value for `key`, or `null` if absent.
`remove(key)`	`key: Key`	`Value`	Removes mapping for `key`, returns its value or `null`.
`containsKey(key)`	`key: Key`	`Boolean`	Returns `true` iff `key` is in the map.
`containsValue(value)`	`value: Value`	`Boolean`	Returns `true` iff `value` is mapped by some key.
`keySet()`	`none`	`Set<Key>`	Returns a set view of all keys.
`values()`	`none`	`Collection<Value>`	Returns a collection view of all values.
`isEmpty()` / `size()`	`none`	`Boolean/Integer`	Standard collection queries.

1.9 Tree ADT — Hierarchical Structure

A **Tree** is a non-linear ADT representing a hierarchy. We will study binary trees in depth in Section 2. Here is the general Tree ADT interface.

Operation	Parameters	Return	Description
`root()`	none	`Node`	Returns the root node (or `null` if empty).
`parent(node)`	`node: Node`	`Node`	Returns parent of `node` (`null` for root).
`children(node)`	`node: Node`	`List<Node>`	Returns list of children of `node`.
`isInternal(node)`	`node: Node`	`Boolean`	True if `node` has at least one child.
`isExternal(node)` / `isLeaf(node)`	`node: Node`	`Boolean`	True if `node` has no children.
`size()`	none	`Integer`	Number of nodes in the tree.
`isEmpty()`	none	`Boolean`	True if tree has no nodes.

1.10 Specifying ADTs: Pre-conditions, Post-conditions, and Invariants

To describe an ADT rigorously, we use **pre-conditions**, **post-conditions**, and **invariants**. This style of specification is examinable and mirrors formal methods used in software engineering.

DEFINITION **Pre-condition, Post-condition, Invariant**

- A pre-condition is a predicate that must be true before an operation is called. If violated, the operation's behaviour is undefined (typically an exception is raised).
- A post-condition is a predicate that must be true after the operation completes (assuming the pre-condition held).
- An invariant is a predicate that is true before and after every public operation of the ADT. It characterises the valid states of the data structure.

METHOD **Writing an ADT Specification**

To specify an ADT operation formally:

1. Name and signature: Give the operation name, parameter types, and return type.
2. Pre-condition: State what must be true before the call (e.g., `!isEmpty()`).
3. Post-condition: State what is true after the call, relating the new state to the old state.

Use `old` notation for previous values (e.g., `size() = old.size() + 1`).

4. Exceptions: List what happens if the pre-condition is violated (e.g., `throw EmptyStackException`).
5. Invariant (optional but recommended): State the property that always holds (e.g., `0 <= top < capacity` for an array stack).

EXEMPLE **Formal Specification of Stack ADT Operations**

ADT: Stack of Elements
State: A finite sequence $S = \langle e_1, e_2, \ldots, e_n \rangle$ where e_n is the top. n = `size()`.
Invariant: $n \geq 0$. (If bounded: $n \leq$ `CAPACITY`.)

Operation	Specification
`push(item)`	Pre: $n <$ `CAPACITY` (if bounded). Post: $S' = \langle e_1, \ldots, e_n,$ `item`$\rangle$; `size()`$' = n + 1$.
`pop()`	Pre: $n > 0$ (i.e., `!isEmpty()`). Post: Returns e_n; $S' = \langle e_1, \ldots, e_{n-1} \rangle$; `size()`$' = n - 1$.
`peek()`	Pre: $n > 0$. Post: Returns e_n; $S' = S$ (state unchanged).
`isEmpty()`	Pre: `true` (always). Post: Returns $(n = 0)$; state unchanged.

EXEMPLE **Formal Specification of Queue ADT Operations**

ADT: Queue of Elements
State: A finite sequence $Q = \langle e_1, e_2, \ldots, e_n \rangle$ where e_1 is the front, e_n is the rear.
Invariant: $n \geq 0$.

Operation	Specification
`enqueue(item)`	Pre: $n <$ `CAPACITY` (if bounded). Post: $Q' = \langle e_1, \ldots, e_n,$ `item`$\rangle$; `size()`$' = n + 1$.
`dequeue()`	Pre: $n > 0$. Post: Returns e_1; $Q' = \langle e_2, \ldots, e_n \rangle$; `size()`$' = n - 1$.
`front()`	Pre: $n > 0$. Post: Returns e_1; $Q' = Q$.

1.11 Worked Example: Specifying a Bounded Stack ADT in Pseudocode

WORKED EXAMPLE 1 **Complete ADT Specification for a Bounded Stack**

Problem: Write a complete ADT specification for a `BoundedStack` of integers with a fixed capacity `MAX`. Include the data structure (hidden), the invariant, and full pre/post conditions for each operation. Use the pseudocode style expected in GCE examinations.

SOLUTION

Solution:

ADT BoundedStack

Data Structure (Hidden):
`items: Array[0..MAX-1] of Integer`
`top: Integer` // index of top element, -1 means empty

Invariant:
`0 <= top+1 <= MAX` // size = top+1
`items[0..top]` hold the stack elements, bottom at index 0.

Operations:

`procedure push(item: Integer)`
Pre: `top < MAX-1` // stack not full
Post: `top = old.top + 1` and `items[top] = item`
Exception: if `top = MAX-1` then raise StackOverflowException

`function pop(): Integer`
Pre: `top >= 0` // stack not empty
Post: `result = items[old.top]` and `top = old.top - 1`
Exception: if `top < 0` then raise EmptyStackException

`function peek(): Integer`
Pre: `top >= 0`
Post: `result = items[top]` and `top = old.top` // state unchanged
Exception: if `top < 0` then raise EmptyStackException

`function isEmpty(): Boolean`
Pre: `true`
Post: `result = (top = -1)`

`function size(): Integer`
Pre: `true`
Post: `result = top + 1`

`function isFull(): Boolean`
Pre: `true`
Post: `result = (top = MAX-1)`

Examiner's Notes:

- The Data Structure section shows the concrete representation — this is not part of the ADT visible to the client, but it is required when you are asked to implement the ADT.
- The Invariant uses the hidden variables. It must hold after construction and after every public operation.
- `old.top` denotes the value of `top` before the operation. This is standard notation in formal specifications.

- Exceptions are explicitly stated. In GCE pseudocode, write `raise ExceptionName` or `throw ExceptionName`.

1.12 Why ADTs Matter: Modularity and Maintainability

KEY POINTS **Key Points: Benefits of ADTs**

1. Abstraction: Clients reason about what the data structure does, not how. This reduces cognitive load.
2. Interchangeability: You can replace an `ArrayStack` with a `LinkedStack` without changing a single line of client code, provided both implement the same `Stack` interface.
3. Team Development: One programmer designs the ADT interface; another implements it. They work in parallel.
4. Testing: You can test client code against a mock implementation, then swap in the real one.
5. Maintenance: Bug fixes or performance improvements in the implementation do not affect clients.

DID YOU KNOW? **ADTs in Real Cameroonian Systems**

Mobile money platforms (like MTN MoMo or Orange Money) use Queues to process transaction requests in order, Maps to look up user balances by phone number (key), and Stacks to implement "undo" functionality in USSD menus. The core banking software hides all these behind clean ADT interfaces so that when the engineering team optimises the queue from a simple array to a priority queue (for VIP customers), the USSD menu code does not need to change.

1.13 Common Pitfalls and Examination Tips

WARNING **Examination Traps**

1. Confusing ADT vs. Data Structure: "Describe the Stack ADT" ≠ "Describe how to implement a stack using an array." The first asks for operations and semantics; the second asks for variables and algorithms.

2. Forgetting Pre-conditions: Always state pre-conditions for `pop`, `peek`, `dequeue`, `front`, `get`, `remove`. An operation without a pre-condition is incomplete.
3. Ignoring Exceptions: If a pre-condition can be violated, say what happens (raise exception, return error code, etc.). "Undefined behaviour" is not an acceptable answer.
4. Wrong Invariant: The invariant must be expressible in terms of the hidden data structure variables, not the abstract state. For a linked-list stack, the invariant might be `head = null` $\iff$ `size = 0`.
5. Off-by-one in Bounded Structures: For an array of size `MAX`, valid indices are `0..MAX-1`. `top = -1` means empty; `top = MAX-1` means full. Check your boundary conditions.

EXERCISE 1 **Practice: ADT Specification**

1. Write the full ADT specification (invariant, pre/post conditions) for a Bounded Queue implemented with a circular array of capacity `MAX`. Use hidden variables `items[0..MAX-1]`, `front`, `rear`, `count`.
2. A Set ADT is implemented using a sorted array (no duplicates). Write the invariant and the specification for `add(item)` and `contains(item)`.
3. Explain why a `List` ADT with operations `insert(pos, item)` and `remove(pos)` cannot be efficiently implemented with a singly linked list if the client frequently accesses positions near the end. What data structure would you choose instead?
4. (GCE-style) A stack is used to evaluate postfix expressions. Given the postfix expression `5 3 2 * + 4 -`, trace the stack contents after each token is processed. What is the final result?

SOLUTION **Exercise 1**

Bounded Circular Queue ADT Specification
Data Structure (Hidden):

- `items: Array[0..MAX-1] of Element`
- `front: Integer` // index of front element
- `rear: Integer` // index where next element will be inserted
- `count: Integer` // number of elements in queue

Invariant:

- $0 \leq$ `count` $\leq$ `MAX`
- If `count` = 0: `front` and `rear` are undefined (or equal, by convention).

- If `count > 0`: elements are stored in `items[front], items[(front+1)%MAX], …, items[(front+count-1)%MAX]`.
- `rear = (front + count) % MAX`

Operations:

- `enqueue(item)`: Pre: `count < MAX`. Post: `items[rear] = item; rear = (rear+1)%MAX; count = count+1`. Exception: `QueueOverflow`.
- `dequeue()`: Pre: `count > 0`. Post: `result = items[front]; front = (front+1)%MAX; count = count-1`. Exception: `EmptyQueue`.
- `front()`: Pre: `count > 0`. Post: `result = items[front]`; state unchanged.
- `isEmpty()`: Post: `result = (count = 0)`.
- `isFull()`: Post: `result = (count = MAX)`.
- `size()`: Post: `result = count`.

SOLUTION **Exercise 2**

Set via Sorted Array

Data Structure: `elements: Array[0..MAX-1] of Element` (sorted ascending), `size: Integer`.

Invariant: $0 \le$ `size` $\le$ `MAX`; `elements[0..size-1]` are strictly increasing (no duplicates).

Operations:

- `add(item)`: Pre: `size < MAX`. Post: If `item` not in `elements[0..size-1]`, it is inserted at the correct position to maintain sorted order, all larger elements shifted right, `size = old.size+1`, returns `true`. If already present, state unchanged, returns `false`. (Binary search for position: $O(\log n)$; shift: $O(n)$.)
- `contains(item)`: Pre: `true`. Post: Returns `true` iff `item` found in `elements[0..size-1]` (binary search, $O(\log n)$). State unchanged.

SOLUTION **Exercise 3**

A singly linked list only allows forward traversal from the head. Accessing position k takes $O(k)$ time. If the client frequently inserts/removes near the end (large k), each operation takes $O(n)$ time just to find the position. A doubly linked list with a `tail` pointer allows $O(1)$ access to the end, but still $O(k)$ for arbitrary positions. For efficient arbitrary positional access, a dynamic array (e.g., `ArrayList`) gives $O(1)$ `get/set` and amortized $O(1)$ `add` at end, though `insert/remove` in the middle remain $O(n)$ due to shifting. A balanced binary search tree with size fields in nodes (order-statistic tree) gives $O(\log n)$ for all operations.

SOLUTION **Exercise 4**

Postfix Evaluation Trace: `5 3 2 * + 4 -`

Token	Action	Stack (top on right)
5	push 5	`[5]`
3	push 3	`[5, 3]`
2	push 2	`[5, 3, 2]`
*	pop 2, pop 3, push 3 × 2 = 6	`[5, 6]`
+	pop 6, pop 5, push 5 + 6 = 11	`[11]`
4	push 4	`[11, 4]`
-	pop 4, pop 11, push 11 - 4 = 7	`[7]`

Final result: 7.

1.14 Summary

In this section, we have established the foundation of Abstract Data Types:

- An **ADT** defines a set of values and operations independently of implementation.
- The six core ADTs — **Stack, Queue, List, Set, Map, Tree** — each have a characteristic set of operations and semantic rules (LIFO, FIFO, positional, unique keys, hierarchy).
- Rigorous specification uses **pre-conditions**, **post-conditions**, and **invariants**.
- **Encapsulation** separates interface from implementation, enabling modular, maintainable, and testable code.
- In examinations, distinguish clearly between **specifying the ADT** (what it does) and **implementing it** (how it stores data and executes operations).

In the next section, we will dive deep into **Binary Tree Concepts and Representation**, where the hierarchical Tree ADT takes concrete form.

2 BINARY TREE CONCEPTS AND REPRESENTATION

In the previous section we met **abstract data types** such as the stack, the queue and the list, and we saw that an ADT is defined by its operations, not by its implementation. All of those structures are linear: each element has at most one successor. But many real situations are hierarchical: the file system on a school computer, the administrative chart of a college (principal → vice-principals → heads of department → teachers), the classification of living things, or the draw of a knockout football tournament like the Cameroon Cup. None of these fit naturally into a line. In this section we study the most important hierarchical structure at A-Level: the **binary tree**.

2.1 Tree Terminology

DEFINITION **Tree**

A **tree** is a finite set of **nodes** connected by **edges** such that:

- there is exactly one distinguished node called the **root**, which has no parent;
- every other node has exactly one **parent**;
- there are no cycles (following edges from the root, you can never return to a node already visited).

The vocabulary of trees is borrowed from family trees and from botany. Fix it precisely, because GCE questions test it directly:

- **Node**: an element of the tree holding data (and, in a linked implementation, pointers).
- **Edge**: a link between a parent and a child.
- **Root**: the unique top node (no parent).
- **Parent** / **child**: node P is the parent of node C if an edge goes from P down to C; C is a child of P.
- **Siblings**: nodes sharing the same parent.
- **Leaf** (external node): a node with no children. Nodes that are not leaves are **internal nodes**.
- **Subtree**: any node together with all its descendants forms a tree in its own right.
- **Depth** of a node: the number of edges from the root to that node (the root has depth 0).
- **Level**: the set of all nodes having the same depth.
- **Height** of a tree: the number of edges on the longest downward path from the root to a leaf (equivalently, the maximum depth). A tree with a single node has height 0.

WARNING **Depth vs. height**

Depth is measured from the root down to a node; height is measured from a node (or the root) down to the furthest leaf. Examiners love to swap them. Also note the convention used here: height counts edges, so a single node has height 0, not 1. State your convention in the exam if the question is ambiguous.

2.2 Binary Trees: Definition and Special Forms

DEFINITION **Binary Tree**

A **binary tree** is a tree in which every node has at most two children, distinguished as the **left child** and the **right child**. The order matters: a node with only a left child is not the same tree as a node with only a right child.

Several special shapes appear constantly in examination questions:

- **Full** (strict) binary tree: every node has 0 or 2 children — never exactly 1.
- **Complete** binary tree: every level is completely filled, except possibly the last, which is filled from left to right with no gaps. (This is exactly the shape of a heap.)
- **Perfect** binary tree: all internal nodes have two children and all leaves are at the same level.
- **Balanced** binary tree: for every node, the heights of its left and right subtrees differ by at most 1 (informally: the tree stays "bushy", so its height remains small, about $\log_2 n$).
- **Binary search tree** (BST): for every node, all keys in its left subtree are **smaller** than the node's key, and all keys in its right subtree are **greater**. This ordering property makes searching fast.

PROPERTY **Maximum number of nodes**

A binary tree of height h contains at most $2^{h+1} - 1$ nodes (and exactly 2^i nodes on level i in a perfect tree).

Justification (instructive, not required for reproduction). Level i can hold at most twice as many nodes as level $i - 1$, and level 0 holds 1 node, so level i holds at most 2^i nodes. Summing the geometric progression over levels 0 to h:

$$1 + 2 + 4 + \cdots + 2^h = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1.$$

Conversely, a tree with n nodes has height at least $\lceil \log_2(n+1) \rceil - 1$ (balanced case) and at most $n - 1$ (degenerate "chain" case).

EXEMPLE **Counting**

A perfect binary tree of height 3 has $2^4 - 1 = 15$ nodes: 1 root, 2 on level 1, 4 on level 2, 8 leaves on level 3.

2.3 Array (Heap-Style) Representation

A complete binary tree can be stored very compactly in an array, without any pointers. This is the representation used by the heap you will meet later. Using **1-based indexing** (root at index

1), the nodes are stored level by level, left to right, and:

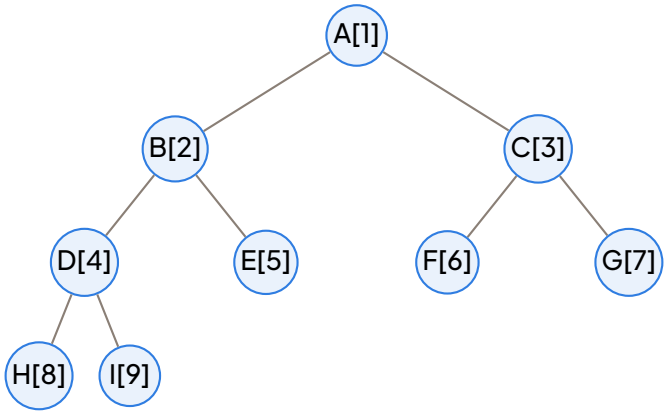
PROPERTY **Array index formulas (1-based)**

For the node stored at index i:

- its parent is at index $\lfloor i/2 \rfloor$;
- its left child is at index $2i$;
- its right child is at index $2i + 1$.

WARNING **Off-by-one errors**

These formulas assume the root is at index 1. If your array is 0-based (root at index 0), the formulas become: parent $\lfloor (i - 1)/2 \rfloor$, left child $2i + 1$, right child $2i + 2$. Mixing the two conventions is the single most common source of wrong answers in trace questions. Always write down which convention you are using before computing.



A binary tree with each node labelled by its data and its 1-based array index in brackets. The array stores: index 1:A, 2:B, 3:C, 4:D, 5:E, 6:F, 7:G, 8:H, 9:I.

EXEMPLE **Using the formulas**

In the tree above, node E is at index 5: its parent is at $\lfloor 5/2 \rfloor = 2$ (node B), its left child would be at $2 \times 5 = 10$ and its right child at 11 — both absent, so E is a leaf.

The array representation is memory-efficient for complete trees but wasteful for sparse ones (a degenerate tree of height h would need an array of size about 2^{h+1}).

2.4 Linked Representation

The most flexible representation uses one record per node:

```
TYPE TreeNode = RECORD
  data : ElementType
  left : POINTER TO TreeNode
  right : POINTER TO TreeNode
END RECORD
```

A whole tree is then simply a pointer to its root; an empty tree is the null pointer. This structure is naturally recursive: the left and right fields of a node point to the roots of its left and right subtrees. That is why recursive algorithms fit binary trees so well.

2.5 Traversal Algorithms

To **traverse** a tree means to visit every node exactly once, in a systematic order. Three recursive depth-first orders and one breadth-first order are required.

METHOD **Recursive in-order traversal**

For a BST, in-order traversal visits the keys in ascending order — a fact frequently examined.

1. Traverse the left subtree (recursively).
2. Process (visit) the current node.
3. Traverse the right subtree (recursively).

```
PROCEDURE InOrder(node)
  IF node ≠ NULL THEN
    InOrder(node.left)
    OUTPUT node.data
    InOrder(node.right)
  END IF
END PROCEDURE
```

The other two depth-first orders differ only in the position of the visit:

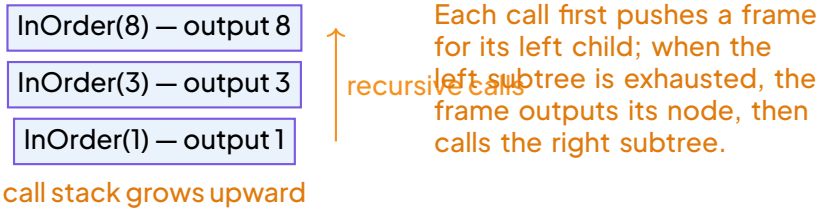
- **Pre-order** (node, left, right): useful to copy a tree or to print a directory structure.
- **Post-order** (left, right, node): useful to delete a tree (children freed before the parent) or to evaluate an expression tree.

WORKED EXAMPLE 2 **Traversals of a small tree**

Consider the BST with root 8, left child 3 (with children 1 and 6), right child 10 (with right child 14).

SOLUTION

- Pre-order: 8, 3, 1, 6, 10, 14
- In-order: 1, 3, 6, 8, 10, 14 — sorted, as expected for a BST.
- Post-order: 1, 6, 3, 14, 10, 8



Snapshot of the call stack during in-order traversal of the BST {8, 3, 1, 6, 10, 14}, at the moment node 1 is being output.

METHOD **Iterative level-order traversal (breadth-first) using a queue**

1. Create an empty queue Q; enqueue the root.
2. While Q is not empty:
   (a) Dequeue a node and process it.
   (b) Enqueue its left child (if it exists), then its right child (if it exists).

The queue guarantees that nodes are visited level by level, left to right.

```
PROCEDURE LevelOrder(root)
  Q ← empty queue; Enqueue(Q, root)
  WHILE NOT IsEmpty(Q) DO
    node ← Dequeue(Q)
    OUTPUT node.data
    IF node.left ≠ NULL THEN Enqueue(Q, node.left)
```

```
    IF node.right ≠ NULL THEN Enqueue(Q, node.right)
  END WHILE
END PROCEDURE
```

For the worked-example BST, level-order gives 8, 3, 10, 1, 6, 14.

2.6 Operations on a Binary Search Tree

Search. Compare the target key with the current node: if equal, stop; if smaller, go left; if greater, go right; if you fall off the tree (NULL), the key is absent.

```
FUNCTION Search(node, key)
  IF node = NULL OR node.data = key THEN RETURN node
  IF key < node.data THEN RETURN Search(node.left, key)
  ELSE RETURN Search(node.right, key)
END FUNCTION
```

Insertion. Search for the key; when you reach a NULL position, attach the new node there (duplicates are usually rejected or sent consistently to one side).

```
PROCEDURE Insert(VAR node, key)
  IF node = NULL THEN node ← new TreeNode(key, NULL, NULL)
  ELSE IF key < node.data THEN Insert(node.left, key)
  ELSE IF key > node.data THEN Insert(node.right, key)
END PROCEDURE
```

Deletion has three cases for the node *N* to remove:

1. *N* is a **leaf**: simply detach it (set the parent's pointer to NULL).
2. *N* has **one child**: replace *N* by that child.
3. *N* has **two children**: find *N*'s in-order successor — the smallest node in its right subtree (go right once, then left as far as possible) — copy its key into *N*, then delete that successor (which has at most one child, so case 1 or 2 applies).

WARNING **BST deletion traps**

- In the two-children case, do not delete the node and try to reattach both subtrees; replace by the in-order successor (or predecessor).
- After deletion, the BST property must still hold for every node — check the whole affected path, not just the deleted node.

2.7 Complexity: Balanced vs. Degenerate Trees

The cost of search, insertion and deletion is proportional to the number of nodes visited on one root-to-leaf path, i.e. to the height h of the tree.

Shape	Height for n nodes	Cost of search / insert / delete
Balanced (e.g. complete)	$h \approx \log_2 n$	$O(\log n)$
Degenerate (chain, e.g. inserting sorted data into a BST)	$h = n - 1$	$O(n)$

For example, with $n = 1\,000\,000$ records, a balanced BST needs about $\log_2 10^6 \approx 20$ comparisons per search, while a degenerate tree may need up to a million — no better than an unsorted list. This is why keeping trees balanced matters in practice (AVL trees, red-black trees), and why level-order traversal, which uses a queue, costs $O(n)$ time and up to $O(n)$ extra space for the widest level.

EXERCISE 2 **Practice**

1. A perfect binary tree has 63 nodes. What is its height? How many leaves does it have?
2. In a 1-based array representation, the node at index 13 — what are the indices of its parent, left child and right child?
3. Insert the keys 50, 30, 70, 20, 40, 60, 80 in this order into an empty BST. Draw the tree, then give its pre-order, in-order, post-order and level-order traversals. Is the tree balanced?
4. From your tree, delete 70 using the in-order successor method. Draw the result.

SOLUTION **Exercise n°1**

1. $63 = 2^{h+1} - 1 \Rightarrow 2^{h+1} = 64 \Rightarrow h = 5$. Leaves $= 2^5 = 32$.
2. Parent: $\lfloor 13/2 \rfloor = 6$; left child: $2 \times 13 = 26$; right child: 27.
3. The tree is perfect/balanced of height 2: root 50; children 30, 70; grandchildren 20, 40, 60, 80. Pre-order: 50, 30, 20, 40, 70, 60, 80. In-order: 20, 30, 40, 50, 60, 70, 80. Post-order: 20, 40, 30, 60, 80, 70, 50. Level-order: 50, 30, 70, 20, 40, 60, 80. Yes, balanced.
4. The in-order successor of 70 is 80 (right child, no left subtree). Copy 80 into the node, then delete the leaf 80: root 50, right child 80, whose left child is 60.

KEY POINTS **Key points**

- A binary tree node has at most two ordered children; master the vocabulary (root, leaf, depth, height, level).
- Maximum nodes at height h: $2^{h+1} - 1$; a balanced tree with n nodes has height $\approx \log_2 n$.
- Array representation (1-based): parent $\lfloor i/2 \rfloor$, left $2i$, right $2i + 1$ — beware 0-based conversions.
- Linked representation: node = data + left pointer + right pointer; recursion fits naturally.
- Traversals: pre-order (NLR), in-order (LNR — sorted for a BST), post-order (LRN), level-order (queue).
- BST operations (search, insert, delete) cost $O(h)$: $O(\log n)$ if balanced, $O(n)$ if degenerate.

In the next section we put these structures to work: we will see how stacks, queues, trees and other ADTs combine to solve real problems — expression evaluation, priority scheduling, and more — which is exactly the kind of synthesis the GCE Paper expects.

3 APPLICATIONS OF ADTS

Having explored the internal structure and representation of binary trees, we now turn to the practical power of Abstract Data Types. An ADT is a contract: it specifies what operations are available, not how they are implemented. This separation allows us to choose the most efficient concrete structure (array, linked list, tree, hash table) for a given problem. In this section we examine classic applications of stacks, queues, lists, sets, maps and trees, and we build two case studies that combine several ADTs.

3.1 Stack Application: Expression Evaluation

Arithmetic expressions written in infix notation (e.g. `3 + 4 * 2`) are natural for humans but awkward for computers because operator precedence and parentheses dictate the order of evaluation. A stack solves this elegantly.

PROPERTY **Operator Precedence and Associativity**

For the basic arithmetic operators:

Operator	Precedence	Associativity
*, /	High (2)	Left
+, -	Low (1)	Left

Higher precedence operators are applied before lower precedence ones. Operators of equal precedence are evaluated left-to-right (left-associative) except exponentiation (right-associative), which is not required for the GCE syllabus.

PROPERTY **Stack LIFO and Operator Ordering**

The Last-In-First-Out behaviour of a stack guarantees that when we convert an infix expression to postfix (Reverse Polish Notation), operators are output in the exact order required for correct evaluation. The most recently seen operator of higher or equal precedence is applied before earlier ones.

Shunting-Yard Algorithm (Dijkstra). The algorithm reads tokens left-to-right, uses an operator stack and an output queue.

METHOD **Infix-to-Postfix Conversion**

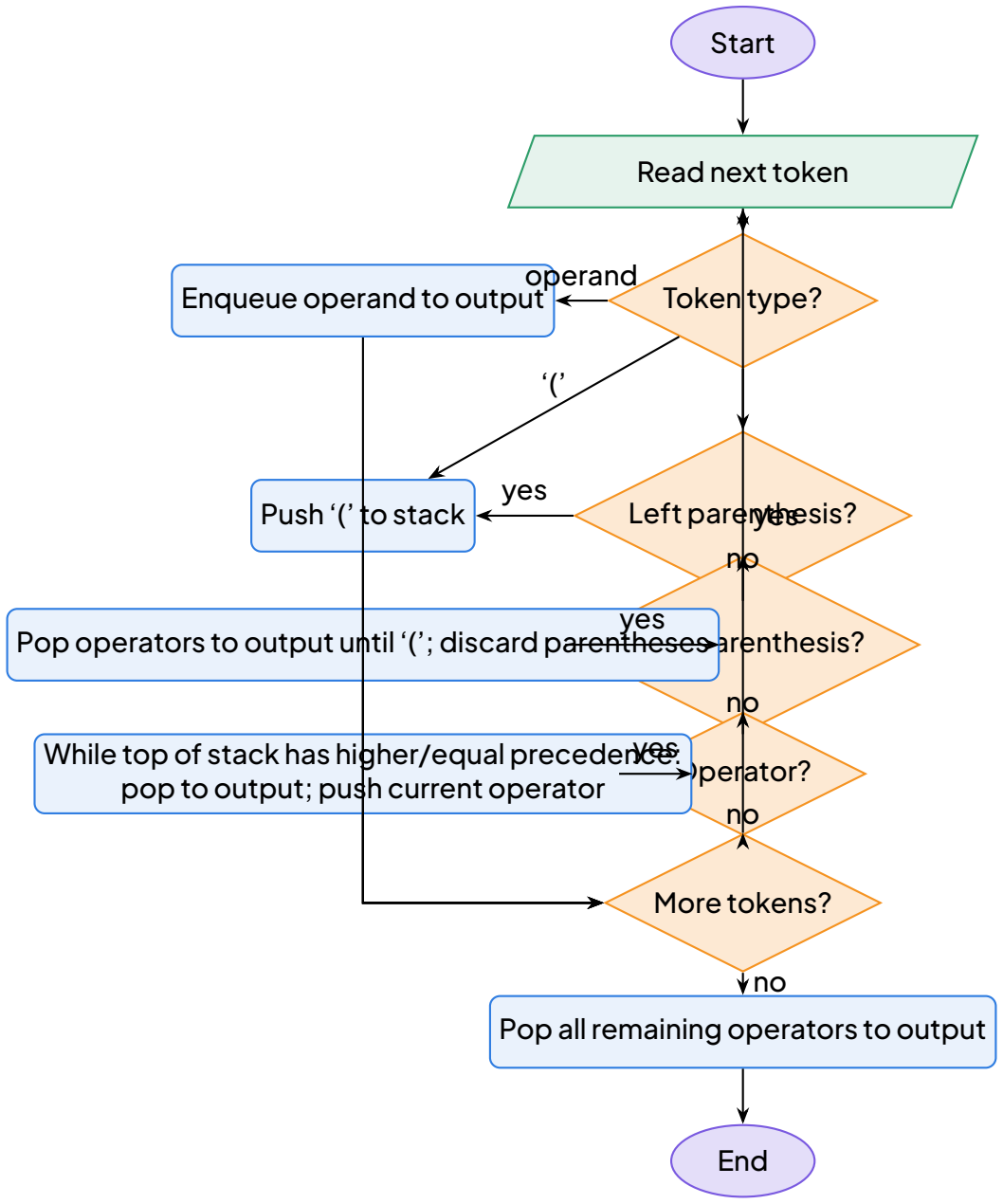
1. Initialise empty operator stack S and output queue Q.
2. For each token t in the infix expression:
   - If t is an operand, enqueue t to Q.
   - If t is a left parenthesis '(', push t onto S.
   - If t is a right parenthesis ')': Pop operators from S to Q until a '(' is found; discard the parentheses.
   - If t is an operator op: While S is not empty and top of S is an operator with higher precedence, or equal precedence and op is left-associative, pop top of S to Q. Push op onto S.
3. After all tokens, pop any remaining operators from S to Q.
4. Q now holds the postfix expression.

EXEMPLE **Converting** `3+4*2/(1-5)`

We trace the algorithm step by step.

Token	Action	Stack *S* (top right)		Output *Q*
3	operand → output	[]		3
+	push +	[+]		3
4	operand → output	[+]		3 4
*	top is + (lower prec) → push *	[+ *]		3 4
2	operand → output	[+ *]		3 4 2
/	top is * (same prec, left-assoc) → pop * to output, push /	[+ /]		3 4 2 *
(	push (	[+ / (]		3 4 2 *
1	operand → output	[+ / (]		3 4 2 * 1
-	push -	[+ / (-]		3 4 2 * 1
5	operand → output	[+ / (-]		3 4 2 * 1 5
)	pop until (: pop - to output, discard (	[+ /]		3 4 2 * 1 5 -
end	pop remaining: / then +	[]		3 4 2 * 1 5 - / +

Postfix: `3 4 2 * 1 5 - / +`.



Flowchart of the Shunting-Yard algorithm for `3+4*2/(1-5)`

Postfix Evaluation. Once we have postfix, evaluation uses a single operand stack.

METHOD **Postfix Evaluation**

1. Initialise empty operand stack S.
2. For each token t in postfix:

- If t is an operand, push t onto S.
- If t is an operator op: Pop b, then a from S (a is left operand, b right). Compute $a\ op\ b$, push result onto S.

3. Final result is the only value left on S.

WORKED EXAMPLE 3 **Evaluate** `3 4 2 * 1 5 - / +`

1. Push 3, 4, 2. Stack: `[3, 4, 2]` (top right).
2. '*': pop 2, 4 → 4*2=8, push 8. Stack: `[3, 8]`.
3. Push 1, 5. Stack: `[3, 8, 1, 5]`.
4. '-': pop 5, 1 → 1−5=−4, push −4. Stack: `[3, 8, -4]`.
5. '/': pop −4, 8 → 8/(−4)=−2, push −2. Stack: `[3, -2]`.
6. '+': pop −2, 3 → 3+(−2)=1, push 1. Stack: `[1]`.
7. Result = 1.

SOLUTION

The algorithm correctly respects precedence: multiplication and division before addition, parentheses first.

WARNING **Stack Overflow**

Recursive expression evaluation or deeply nested parentheses can exhaust the call stack or the explicit operator stack. In production code, set a maximum stack depth (e.g. 1000) and report an error if exceeded. Iterative Shunting-Yard avoids recursion limits.

WARNING **Integer vs Floating-Point**

If the language uses integer division, `5/2` yields 2. For a general calculator, use floating-point (double) throughout, or detect decimal points in tokenisation.

3.2 Queue Application: Scheduling and Breadth-First Search

A queue's First-In-First-Out discipline models any system where order of arrival determines order of service.

- **Print Job Spooler.** Jobs are enqueued as users submit them; the printer dequeues and prints in the same order. Priority queues (a variant) can handle urgent jobs.
- **Breadth-First Search (BFS) in Graphs.** Starting from a source vertex, we enqueue it. While the queue is not empty, dequeue a vertex, visit its unvisited neighbours and enqueue them. This guarantees shortest-path (in edges) discovery in unweighted graphs.
- **Waiting-Line Simulation.** Customers arrive at random intervals (Poisson process), join a queue, are served by one or more servers (exponential service time). Queue length and waiting time statistics inform staffing decisions (e.g. at a bank or a hospital triage).

EXEMPLE **BFS Pseudocode**

```
queue ← empty
enqueue(queue, start)
visited[start] ← true
while not empty(queue):
  u ← dequeue(queue)
  for each neighbour v of u:
    if not visited[v]:
      visited[v] ← true
      enqueue(queue, v)
```

3.3 List Application: Dynamic Collections

Lists (array-based or linked) support insertion and deletion at any position, making them ideal for ordered collections that change frequently.

EXEMPLE **Student Record Management with a List**

A school maintains an alphabetical list of students. Each record: `(ID, Name, Form, Average)`.

- Insert: Find correct position (binary search on name if array list, linear if linked), shift elements (array) or relink nodes (linked), insert new record.
- Delete: Locate record by ID, remove, shift/relink.
- Update: Locate, modify fields (e.g. new average after exams).
- Report: Traverse list in order to print termly report cards.

A linked list avoids costly shifting for large datasets; an array list gives cache-friendly traversal for reporting.

3.4 Set Application: Uniqueness and Membership

A set stores distinct elements with fast membership testing.

- **Removing Duplicates from a Data Stream.** Read each item; if not in set, add to set and write to output. Guarantees each value appears once.
- **Membership Testing.** Check if a student ID exists in the set of registered candidates for an exam. Hash-set gives $O(1)$ average time.

EXEMPLE **Duplicate Removal in a Log File**

A server log contains IP addresses. To count unique visitors:

```
unique_ips ← empty hash set
for each line in log:
  ip ← extract_ip(line)
  if ip not in unique_ips: add ip to unique_ips
print size(unique_ips)
```

3.5 Map (Dictionary) Application: Key-Value Lookup

A map associates a unique key with a value. Operations: `put(key, value)`, `get(key)`, `remove(key)`.

DEFINITION **Symbol Table**

A symbol table is a map used by a compiler or interpreter to store information about identifiers (variables, functions, classes). Keys are identifier names; values are attributes (type, scope, memory address, line numbers). Fast lookup ($O(1)$ average with hash map, $O(\log n)$ with balanced tree) is critical during compilation.

EXEMPLE **Student-ID → Record Lookup**

A school database uses a hash map: `Map<String, StudentRecord>`.

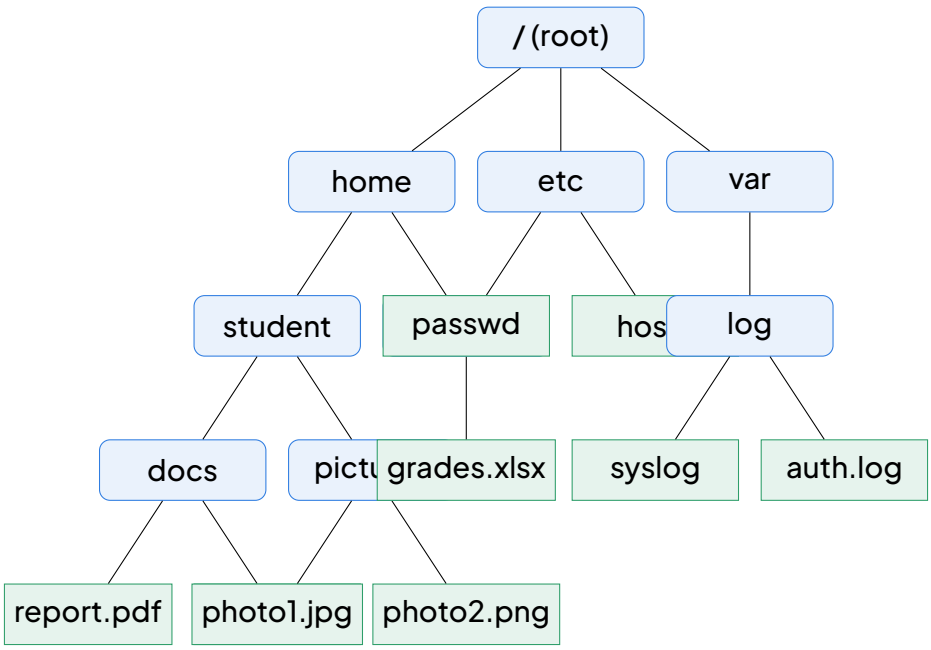
- Key: admission number (e.g. `"2024/SC/00123"`).

- Value: object containing name, form, subjects, grades.
- `get("2024/SC/00123")` retrieves the record in constant time for fee payment, transcript printing, etc.

3.6 Tree Application: Hierarchical Data

Trees naturally represent hierarchies: each node has a parent (except root) and zero or more children.

- **File System.** Directories are internal nodes; files are leaves. Path `/home/student/docs/report.pdf` corresponds to a root-to-leaf traversal.
- **Organisation Chart.** CEO at root, directors as children, managers below, employees as leaves.
- **Decision Tree.** Internal nodes are tests (e.g. `age > 18?`), branches are outcomes, leaves are decisions (e.g. `Eligible`, `Not Eligible`).



File-system tree: directories (blue) and files (green)

3.7 Case Study 1: Simple Calculator with Operator Precedence

We build a calculator that reads an infix expression from the user, converts it to postfix using the Shunting-Yard algorithm, then evaluates the postfix expression. This demonstrates the stack as the central ADT for both phases.

METHOD **Calculator Algorithm**

1. Read input string (e.g. `"3+4*2/(1-5)"`).
2. Tokenise: numbers (multi-digit), operators `+ - * /`, parentheses.
3. Apply Shunting-Yard to produce postfix token list.
4. Evaluate postfix with operand stack.
5. Display result.
6. Handle errors: division by zero, mismatched parentheses, invalid tokens.

WORKED EXAMPLE 4 **Calculator Trace for** `10+20*3`

1. Tokens: `10, +, 20, *, 3`.
2. Postfix: `10 20 3 * +`.
3. Evaluation:
   - Push 10, 20, 3.
   - '*': pop 3, 20 → 60, push 60.
   - '+': pop 60, 10 → 70, push 70.
   - Result = 70.

SOLUTION

The calculator correctly computes 10 + (20 × 3) = 70, not (10 + 20) × 3 = 90.

3.8 Case Study 2: Mini Student-Management System

We combine a **Map** for *O*(1) ID lookup and a **List** for ordered reports (e.g. alphabetical or by merit).

METHOD **System Design**

1. Data Structures:
   - `Map<String, Student> idMap` — key = admission number.
   - `List<Student> alphaList` — kept sorted by name (or use a TreeSet with custom

comparator).

2. Operations:
   - `addStudent(Student s)`: `idMap.put(s.id, s)`; insert into `alphaList` at sorted position.
   - `findById(String id)`: `return idMap.get(id)` — $O(1)$.
   - `removeStudent(String id)`: `Student s = idMap.remove(id);` `alphaList.remove(s)`.
   - `printAlphabetical()`: traverse `alphaList`.
   - `printMeritOrder()`: copy `alphaList` to array, sort by average descending, print.
3. Consistency: Every mutation updates both structures atomically (or within a transaction).

EXEMPLE **Cameroon Context: GCE Registration**

During GCE Advanced Level registration, a centre enters candidates. The map allows instant verification: "Has candidate `2025/AL/00456` already registered?" The list produces the alphabetical nominal roll for the examination board. At the end of term, a merit list (sorted by average) is generated for prize-giving.

3.9 Choosing the Right ADT

METHOD **Choosing an ADT — Decision Guide**

1. Identify the core operations: Insert, delete, search, traverse, access by index, access by key, FIFO, LIFO, hierarchy.
2. Match to ADT strengths:
   - Need LIFO (undo, expression eval) → Stack.
   - Need FIFO (scheduling, BFS) → Queue.
   - Need ordered sequence with frequent insert/delete anywhere → List (linked for many mutations, array for cache-friendly reads).
   - Need uniqueness and fast membership → Set (hash set for $O(1)$, tree set for ordered iteration).
   - Need key-value association with fast lookup → Map (hash map for $O(1)$, tree map for sorted keys).

- Need hierarchy, path queries, prefix search → Tree (general tree, binary search tree, trie).

3. Consider constraints: Memory, latency, concurrency, persistence.
4. Prototype and benchmark: Implement with two candidate structures, test with realistic data sizes.

KEY POINTS **Key Points for Examination**

- Stack: infix→postfix (Shunting-Yard), postfix evaluation, undo mechanisms, recursion simulation.
- Queue: print spooling, BFS, simulation of waiting lines.
- List: dynamic ordered collections (student records, playlists).
- Set: duplicate removal, membership testing, mathematical set operations (union, intersection).
- Map: symbol tables, dictionaries, ID→record lookup, caching.
- Tree: file systems, organisation charts, decision trees, XML/JSON parsing.
- Always justify your ADT choice by the required operations and their complexity.

EXERCISE 3 **Practice Questions**

1. Convert the infix expression `(A+B)*(C-D)/E` to postfix using the Shunting-Yard algorithm. Show the stack and output at each step.
2. A printer receives jobs J1, J2, J3, J4 in that order. J2 is a high-priority job. Explain how a priority queue would change the print order compared to a simple FIFO queue.
3. Design a data structure to store a dictionary of English words for a spell-checker. The operations are: `addWord`, `checkWord`, `suggestCorrections` (words with edit distance 1). Which ADT(s) would you use and why?
4. In the mini student-management system, suppose we also need to quickly find all students in a given form (e.g. Upper Sixth). How would you modify the design?
5. Draw the file-system tree for the following paths: `/etc/passwd`, `/etc/hosts`, `/home/alice/docs/report.pdf`, `/home/bob/music/song.mp3`, `/var/log/syslog`.

SOLUTION **Exercise 1**

Tokens: '(', A, +, B, ')', *, '(', C, -, D, ')', /, E. Trace:

Token	Action	Stack	Output
(	push (	[(]	[]
A	output	[(]	[A]
+	push +	[(+]	[A]
B	output	[(+]	[A B]
)	pop +, pop (	[]	[A B +]
*	push *	[*]	[A B +]
(	push (	[* (]	[A B +]
C	output	[* (]	[A B + C]
-	push -	[* (-]	[A B + C]
D	output	[* (-]	[A B + C D]
)	pop -, pop (	[*]	[A B + C D -]
/	pop * (same prec, left-assoc), push /	[/]	[A B + C D - *]
E	output	[/]	[A B + C D - * E]
end	pop /	[]	[A B + C D - * E /]

Postfix: `A B + C D - * E /`.

DID YOU KNOW? **Did You Know?**

The Shunting-Yard algorithm was invented by Edsger Dijkstra in 1961. Its name comes from the resemblance to a railway shunting yard where trains (tokens) are rearranged from an input track to an output track using a holding siding (the stack). Dijkstra also gave us the shortest-path algorithm and the concept of structured programming.

4 INTEGRATION ACTIVITY & EXAM PREPARATION

In the previous sections we saw how abstract data types appear in real software: queues in print servers, stacks in expression evaluation, trees in file systems, and maps in databases. Each ADT was studied in isolation, with its specification, its representation and its operations. But real problems never come labelled "use a queue here". The true skill of a computer scientist — and the skill the GCE Advanced Level examiner rewards — is to look at a messy, realistic situation and decide which ADTs to combine, and why. That is exactly what an **Integration Activity** trains you to do. This final section brings the whole chapter together: we review every ADT, we

learn a repeatable design methodology, we apply it to a complete mini-project, and we finish with GCE-style practice and a self-assessment checklist.

DEFINITION **Integration Activity**

An **Integration Activity** is a guided project that requires combining several ADTs to solve a realistic problem. It tests not only knowledge of each ADT (specification, representation, operations) but also the higher-order skills of analysing requirements, selecting appropriate structures, and justifying choices in terms of efficiency and clarity.

4.1 Review of All ADTs Covered

Before designing anything, let us consolidate what we know. The table below summarises each ADT studied in this chapter: its specification (what it does), its usual representation (how it is stored), and typical use-cases.

ADT	Specification (key operations)	Common representation	Typical use-cases
Stack	`push`, `pop`, `top`, `isEmpty` — LIFO	Array + top pointer, or linked list	Undo, expression evaluation, function calls
Queue	`enqueue`, `dequeue`, `front`, `isEmpty` — FIFO	Circular array, or linked list with two pointers	Print jobs, process scheduling, reservations
List	`insert`, `delete`, `search`, `traverse`	Array or linked nodes	Simple collections, small datasets
Binary Tree / BST	`insert`, `search`, `delete`, traversals (pre/in/post-order)	Linked nodes (2 pointers) or array (heap indexing)	Sorted indexes, expression trees, dictionaries
Map (Dictionary)	`put(key,value)`, `get(key)`, `remove(key)`	Hash table, or BST of (key, value) pairs	Catalogues, symbol tables, lookups by key

PROPERTY **Time Complexity Summary**

For the standard operations, the expected time complexities are:

- Stack / Queue: `push`/`pop`/`enqueue`/`dequeue` in $O(1)$.
- List: `search` in $O(n)$ (linear scan); `insert` at head in $O(1)$ for a linked list.
- BST: `search`/`insert` in $O(\log n)$ on average (balanced tree), $O(n)$ in the worst case (degenerate tree).
- Map (hash table): `get`/`put` in $O(1)$ on average, $O(n)$ worst case (many collisions).

This summary is examinable: you must be able to state these complexities and use them

to justify an ADT choice.

WARNING **Over-engineering**

A frequent mistake — in exams and in real life — is **over-engineering**: reaching for a complex ADT when a simple one suffices. If a shopkeeper in Bamenda keeps only 15 customers and just needs to process them in arrival order, a queue (or even a plain list) is enough; a balanced BST would add complexity with zero benefit. Rule of thumb: choose the simplest ADT whose operations match the requirements and whose complexity is acceptable for the expected data size.

4.2 A Step-by-Step Design Methodology

How do we move from a problem statement to a working ADT-based solution? Follow this disciplined pipeline: requirements → ADT selection → interface design → pseudocode → test cases.

METHOD **ADT-Driven Design**

1. List the operations needed. Read the problem and extract verbs: “add a book”, “find a member”, “serve the next reservation”. Each verb is a candidate operation.
2. Pick an ADT per group of operations. Match each group to the ADT whose specification fits: LIFO? Stack. FIFO? Queue. Fast key lookup? Map. Sorted order? BST.
3. Write the interface. For each chosen ADT, write the operation signatures with pre-conditions and post-conditions (e.g., `dequeue(Q)` requires Q non-empty).
4. Outline the implementation. Choose a representation (array or linked), write the pseudocode of each operation, then design test cases: normal cases, boundary cases (empty structure), and error cases (dequeue from empty queue).

EXEMPLE **Formal ADT Interface for the Catalogue Map**

As an illustration of Step 3, here is the formal interface for the `CatalogueMap` used in the mini-project below.

- `put(isbn: String, book: BookRecord)`
  - Pre: `isbn` not already in map (or update allowed).
  - Post: Map contains association `isbn ↦ book`.

- `get(isbn: String) : BookRecord`
  - Pre: None.
  - Post: Returns the `BookRecord` associated with `isbn`, or `null` if absent; map unchanged.
- `remove(isbn: String)`
  - Pre: `isbn` exists in map.
  - Post: Association removed; map size decreases by 1.

4.3 Mini-Project: A Library Management System

Let us apply the methodology to a realistic Cameroonian scenario: the municipal library of Buea wants a small computerised system.

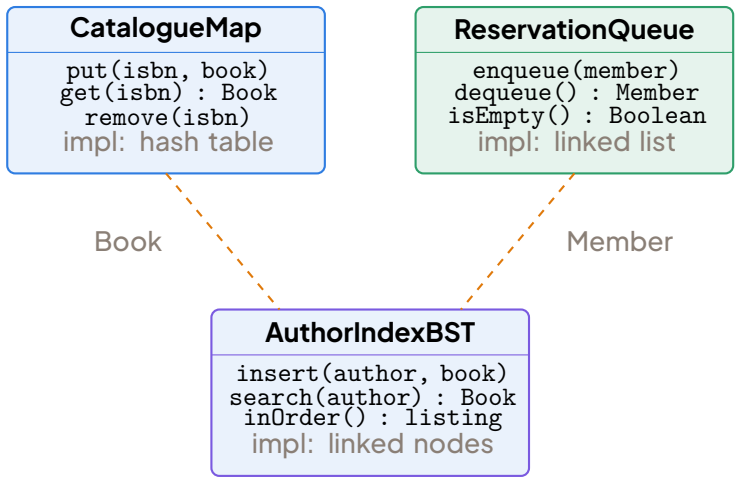
Step 1 — Identify the entities. Reading the requirements, three entities emerge: **Book** (title, author, ISBN, copies), **Member** (name, ID, contact), and **Loan** (which member borrowed which book, due date).

Step 2 — List the operations needed.

- Look up a book instantly by ISBN ⇒ key-based lookup.
- Manage a waiting list when all copies of a popular book (say, a GCE revision guide) are out ⇒ first come, first served.
- Produce an alphabetical listing of books by author ⇒ sorted traversal.

Step 3 — Choose the ADTs.

- A **Map** (hash table) for the catalogue: key = ISBN, value = Book record. Justification: $O(1)$ average lookup.
- A **Queue** for reservations: members wait in arrival order. Justification: fairness is exactly FIFO.
- A **BST** for the author index: books ordered by author name. Justification: in-order traversal gives the alphabetical listing in $O(n)$, and insertion is $O(\log n)$ on average.



Simplified UML-style class diagram of the Library Management System: each class exposes an ADT interface and records its chosen implementation.

Step 4 — Core operation pseudocode.

```
procedure BorrowBook(isbn, member)
  book ← CatalogueMap.get(isbn)
  if book = null then output "Unknown ISBN"
  else if book.copies > 0 then
    book.copies ← book.copies - 1; record Loan(member, book)
  else ReservationQueue.enqueue(member)
procedure ReturnBook(isbn)
  book ← CatalogueMap.get(isbn)
  if not ReservationQueue.isEmpty() then
    next ← ReservationQueue.dequeue(); record Loan(next, book)
  else book.copies ← book.copies + 1
```

Notice how the ADTs cooperate: the Map locates the book, the Queue decides fairly who gets a returned copy, and the BST would produce the end-of-term printed catalogue by author.

Step 5 — Test cases. (i) Borrow an available book: copies decrease by 1. (ii) Borrow an unavailable book: member joins the queue. (iii) Return a book with a non-empty queue: the first waiting member receives it. (iv) Return with an empty queue: copies increase. (v) Unknown ISBN: error message, no state change.

4.4 Common GCE-Style Questions

Certain question types return again and again at the Advanced Level. Master these four patterns.

1. **Trace a traversal.** Given a drawn binary tree, write the pre-order, in-order and post-order sequences. Method: mark each node and apply the rule mechanically (Root-Left-Right, Left-Root-Right, Left-Right-Root).
2. **Convert an expression.** Infix to postfix using a stack (operators wait on the stack according to precedence; operands go straight out), or build an expression tree and read it in post-order.
3. **Justify an ADT choice.** "Which ADT should model X? Justify." Always answer in three parts: name the ADT, match its behaviour to the requirement, and cite the relevant complexity.
4. **Implement a queue with two stacks.** A classic: `enqueue` pushes onto stack S_1; `dequeue` pops from S_2, and if S_2 is empty, first transfer everything from S_1 to S_2 (which reverses the order, restoring FIFO).

WORKED EXAMPLE 5 **Queue with Two Stacks**

Show the state of stacks S_1 and S_2 after: `enqueue(A)`, `enqueue(B)`, `dequeue()`, `enqueue(C)`, `dequeue()`.

SOLUTION

Solution. `enqueue` pushes on S_1; `dequeue` pops from S_2 after transferring if S_2 is empty.

Operation	S_1 (top right)	S_2 (top right)	Output
`enqueue(A)`	A	—	—
`enqueue(B)`	A B	—	—
`dequeue()`	—	B	A
`enqueue(C)`	C	B	—
`dequeue()`	C	—	B

For the first `dequeue`, S_2 is empty, so we transfer S_1: popping B then A from S_1 and pushing them onto S_2 gives S_2 = [B, A] with A on top; popping yields A. The second `dequeue` pops B directly. The queue behaves FIFO: A leaves before B. ■

4.5 Array vs. Linked Representations of Binary Trees

Criterion	Array representation	Linked representation
Memory usage	Wasteful for sparse/unbalanced trees (empty slots)	Exactly one node per element + 2 pointers
Access to parent/children	$O(1)$ via indices ($2i$, $2i+1$, $\lfloor i/2 \rfloor$)	$O(1)$ via pointers (child); parent needs extra pointer
Insertion / deletion	Costly if restructuring is needed	$O(1)$ pointer updates once located
Traversal	Simple index arithmetic	Simple recursion on pointers
Best suited for	Complete trees (heaps, priority queues)	General BSTs, expression trees

4.6 Exam Technique and Self-Assessment

METHOD **Exam Answer Structure**

For any "design" or "justify" question, structure your answer in four moves:

1. State the ADT (and its representation if asked).
2. Give the pseudocode of the key operations, with pre- and post-conditions.
3. Analyse the complexity of each operation.
4. Provide a trace example (a small table or tree diagram showing the structure evolving).

WARNING **Reading Pseudocode Carefully**

In the exam: (i) check whether indices start at 0 or 1; (ii) check whether a queue's "front" pointer indicates the element or the slot before it; (iii) when drawing trees, insert values one at a time in the given order — inserting a whole set at once is the most common cause of wrong BSTs; (iv) always state pre-conditions such as "the stack is not empty" before `pop`.

EXERCISE 4 **Choosing and Justifying ADTs**

A hospital in Douala needs to manage: (a) patients waiting to see a doctor, in arrival order; (b) instant access to a patient's file from their national ID number; (c) an alphabetical printout of all registered patients. For each requirement, choose an ADT, justify it in two sentences including a complexity argument, and give the pseudocode signature of one key operation.

SOLUTION **Exercise 1**

(a) Queue: waiting is first-come-first-served, which is exactly FIFO; `enqueue`/`dequeue` are $O(1)$. Signature: `enqueue(patient)`. (b) Map (hash table): lookup by a unique key (ID number) is $O(1)$ on average, far better than a linear scan in $O(n)$. Signature: `get(id) : PatientFile`. (c) BST keyed on patient name: in-order traversal produces the alphabetical listing in $O(n)$, and each insertion costs $O(\log n)$ on average. Signature: `inOrder(root)`.

EXERCISE 5 **Traversal Trace**

A BST is built by inserting, in this order: 50, 30, 70, 20, 40, 60, 80. Draw the tree and give its in-order, pre-order and post-order traversals.

SOLUTION **Exercise 2**

50 is the root; 30 goes left, 70 right; 20 left of 30, 40 right of 30; 60 left of 70, 80 right of 70. In-order: 20, 30, 40, 50, 60, 70, 80 (sorted, as expected for a BST). Pre-order: 50, 30, 20, 40, 70, 60, 80. Post-order: 20, 40, 30, 60, 80, 70, 50.

KEY POINTS **Self-Assessment Checklist**

Before the examination, confirm that you can:

- Lesson 6: define an ADT, distinguish specification from implementation, and write an interface with pre/post-conditions.
- Lesson 7: define binary tree vocabulary (root, leaf, height, depth), perform the three traversals, and represent a tree with arrays and with linked nodes.
- Lesson 8: match real-life situations (print server, undo, dictionary, file system) to the right ADT and justify the match.
- Lesson 9: run the full design pipeline (requirements → ADT → interface → pseudocode → tests) on an unseen problem, and quote the complexity table from memory.

If every box is ticked, you are ready: you do not merely know the ADTs — you can think with them.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

This integration activity brings together everything you have studied in this chapter: the notion of an **abstract data type** (ADT), the specification of operations with **preconditions** and **postconditions**, the **binary search tree** (BST), and the classic structures **list**, **queue** and **map**. It is deliberately built like a Paper 2 design question: a realistic situation, a requirements document, and a set of tasks moving from analysis to design to tracing. By the end of the activity you should be able to:

- analyse a real requirements document and identify the functional areas of a system;
- **select** an appropriate ADT for each functional area and **justify** the choice in terms of the operations needed and their efficiency;
- **write** formal ADT interfaces (signatures, preconditions, postconditions);
- **design** pseudocode for core operations that compose several ADTs;
- **draw** a class diagram showing the relationships between the classes of the system;
- **trace** a scenario step by step, showing the state of each structure, as expected in Paper 2 of the GCE Advanced Level.

METHOD **How to work**

Work in groups of 3–4. Appoint a secretary (writes the interfaces), an analyst (justifies the ADT choices), a programmer (writes the pseudocode) and a presenter (prepares the 5-minute demonstration). Rotate roles if the activity is done over two sessions. Keep every justification short and technical, in three sentences:

1. one sentence naming the ADT chosen;
2. one sentence naming the operations it offers that the system needs;
3. one sentence on efficiency or ordering (why it beats the obvious alternative).

This three-sentence discipline is exactly the style of justification that earns full marks in the examination.

5.2 Situation

Buea Central Community Library. The Buea Central Community Library, situated near Molyko, serves secondary school students, university students and teachers of the South West Region. Until now, all loans have been recorded in a paper register. The librarian, Mrs. Eposi, complains of three recurring problems:

- when a popular textbook (for example an Advanced Level Physics revision guide) is out on loan, several borrowers queue informally and quarrels arise about “who came first”;

- at the end of each term, she must flip through the whole register by hand to find overdue loans (books kept beyond the 14-day limit) in order to telephone the members concerned;
- searching the catalogue for a book by its title is slow because the cards are not perfectly ordered.

Observe that each complaint points directly at one ADT studied in this chapter: the quarrels point at a **queue** (fair, first-come-first-served), the overdue search points at a **list** that can be scanned, and the slow catalogue points at an ordered structure such as a **binary search tree**. The library committee has obtained a modest grant to computerise the service and has asked your class to produce the design of the new system. The requirements document given to your group reads as follows.

EXEMPLE **Requirements document (extract)**

1. Catalogue. Each book has a unique ISBN (a string), a title, an author and a number of copies. The system must allow a book to be added or removed, and must allow fast look-up of a book by ISBN and an alphabetical listing of titles.
2. Members. Each member has a unique membership number, a name and a telephone number. Members can be registered and removed.
3. Loans. A member may borrow at most 3 books at a time. A loan records the ISBN, the membership number and the date borrowed. The loan period is 14 days. The system must process a borrow request, process a return, and produce the list of overdue loans for a given "today" date.
4. Reservations. If all copies of a book are out on loan, a member may reserve it. Reservations for a given book must be served strictly in arrival order (first come, first served). When a copy is returned, it is offered to the member at the front of that book's reservation queue.

For the demonstration scenario, Mrs. Eposi provides the following small data set.

ISBN	Title	Author	Copies
978-001	Advanced Level Physics Revision	T. Ndi	1
978-002	Cameroon GCE Mathematics	A. Fon	2
978-003	Things Fall Apart	C. Achebe	1

Member no.	Name	Telephone
M01	Ngong Blessing	677 00 11 22
M02	Ekane Samuel	690 33 44 55
M03	Abena Larissa	655 66 77 88

Trace scenario for the demo. Today is day 1. M01 borrows 978-001. M02 borrows 978-001 (it is out, so M02 reserves it). M03 also reserves 978-001. On day 20, M01 returns 978-001:

the copy is offered to M02 (front of the queue), and M03 remains waiting. Also on day 20, the overdue report must show that the loan of 978-001 by M01 (borrowed day 1, due day 15) was overdue from day 16 until its return.

5.3 Tasks

1. **Identify the functional areas.** From the requirements document, list the four functional areas of the system and, for each one, the operations it must support.
2. **Choose and justify the ADTs.** For each functional area, choose one ADT studied in this chapter (List, Queue, Map, BST) and justify your choice in three sentences: the ADT, the operations it provides that you need, and why it fits better than the alternatives. Pay particular attention to:
   (a) the catalogue, which needs fast look-up by ISBN and alphabetical listing;
   (b) the reservations, which must be served first come, first served;
   (c) the link between a book and its reservation queue, and between a member number and the member's record.
3. **Write the ADT interfaces.** Give the interface (name of each operation, parameters, return type, precondition, postcondition) for the two ADTs you consider most delicate: the `ReservationQueue` and the `Catalogue`.
4. **Pseudocode of the core operations.** Write clear pseudocode for:
   (a) `Borrow(isbn, memberNo, today)` — including the check of the 3-book limit and the case where no copy is available (the member is then added to the reservation queue);
   (b) `Return(isbn, today)` — including the offer of the copy to the front of the reservation queue;
   (c) `OverdueReport(today)` — producing the list of overdue loans with member names and telephone numbers.
5. **Class diagram.** Draw a class diagram showing the classes `Book`, `Member`, `Loan`, `Catalogue`, `MemberRegister`, `LoanList` and `ReservationManager`, with the associations between them (for example: the catalogue contains books; a loan links one member and one book).
6. **Demonstration.** Prepare a 5-minute presentation tracing the scenario of the situation. Show, in a trace table, the state of the reservation queue for 978-001 and of the loan list after each event, and state what Mrs. Eposi sees on her screen at each step.

WARNING **Common traps**

- Choosing a stack for reservations: a stack is last-in-first-out, which would serve the last arrival first — exactly the quarrel Mrs. Eposi wants to avoid. Reservations demand a queue (FIFO).
- Forgetting the precondition of `dequeue`: the queue must not be empty. In `Return`, always test `isEmpty()` before offering the copy.

- Confusing the Map (key → value association, e.g. ISBN → reservation queue) with the BST (ordered storage for fast search and sorted listing). Both appear in a good solution, for different jobs.
- In the overdue report, a loan is overdue only if it is still current (not yet returned) and $today - dateBorrowed > 14$. A loan exactly 14 days old is not yet overdue.
- Removing a book from the catalogue while a copy is still out on loan: the precondition of `remove` must forbid this, otherwise the loan list would refer to a book that no longer exists.

5.4 Model answer

Task 1 — Functional areas.

Area	Operations required
Catalogue	add book, remove book, find by ISBN, list titles alphabetically
Members	register member, remove member, find by membership number
Loans	borrow, return, count a member's current loans, list overdue loans
Reservations	reserve a book, serve the longest-waiting member, cancel

Task 2 — Choice and justification of ADTs.

- **Catalogue: a BST keyed on ISBN.** A binary search tree gives `search`, `insert` and `delete` in $O(\log n)$ on average, and an in-order traversal produces the books in sorted key order. If we key the tree on title (or keep a second index), the in-order traversal gives the alphabetical listing Mrs. Eposi needs. A simple list would force an $O(n)$ search for every look-up.
- **Members: a Map (membership number → member record).** A map is exactly the "dictionary" structure: `put`, `get`, `remove` by key. We never need members in sorted order, only direct access by number.
- **Loans: a List.** Loans are processed by scanning (overdue report, counting a member's loans) and there is no natural key for fast look-up; a dynamic list with `add`, `remove` and iteration is sufficient and simple.
- **Reservations: a Queue per book, stored in a Map (ISBN → Queue).** A queue guarantees first-in-first-out service, which is precisely the "first come, first served" rule. The map lets us find the queue of a given book instantly, without scanning all reservations.

Task 3 — ADT interfaces.

EXEMPLE **Interface ReservationQueue (a FIFO queue of membership numbers)**

- `enqueue(m)` — Pre: true. Post: member `m` is added at the rear; size increases by 1.

- `dequeue() → MemberNo` — Pre: the queue is not empty. Post: the front member is removed and returned; size decreases by 1.
- `front() → MemberNo` — Pre: the queue is not empty. Post: returns the front member without removing it.
- `isEmpty() → Boolean` — Pre: true. Post: returns true exactly when size = 0.
- `size() → Integer` — Pre: true. Post: returns the number of waiting members.

EXEMPLE **Interface Catalogue (a BST keyed on ISBN)**

- `insert(book)` — Pre: no book with the same ISBN exists. Post: `book` is stored; the BST ordering property is preserved.
- `remove(isbn)` — Pre: a book with this ISBN exists and has no current loan. Post: the book is deleted; the BST property is preserved.
- `search(isbn) → Book` — Pre: true. Post: returns the book with this ISBN, or `null` if absent.
- `inOrder() → List of Book` — Pre: true. Post: returns all books in ascending key order.

Task 4 — Pseudocode of the core operations.

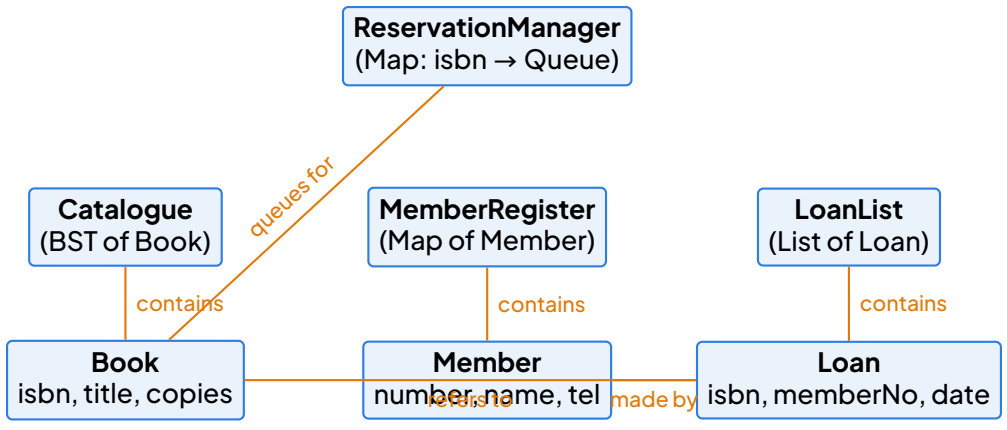
EXEMPLE **Borrow, Return and OverdueReport**

```
PROCEDURE Borrow(isbn, memberNo, today)
  book ← Catalogue.search(isbn)
  IF book = null THEN output "Unknown book"; STOP
  IF LoanList.countFor(memberNo) ≥ 3 THEN
    output "Limit of 3 loans reached"; STOP
  IF book.copiesAvailable > 0 THEN
    book.copiesAvailable ← book.copiesAvailable - 1
    LoanList.add(Loan(isbn, memberNo, today))
  ELSE
    q ← Reservations.get(isbn)   // a Queue
```

```
    q.enqueue(memberNo)
    output "Reserved; position " + q.size()
PROCEDURE Return(isbn, today)
  loan ← LoanList.findCurrent(isbn)
  LoanList.remove(loan)
  book ← Catalogue.search(isbn)
  q ← Reservations.get(isbn)
  IF NOT q.isEmpty() THEN
    next ← q.dequeue()
    LoanList.add(Loan(isbn, next, today))
    output "Copy offered to " + next
  ELSE
    book.copiesAvailable ← book.copiesAvailable + 1
FUNCTION OverdueReport(today) → List
  report ← empty list
  FOR EACH loan IN LoanList DO
    IF today - loan.dateBorrowed > 14 THEN
      m ← Members.get(loan.memberNo)
      report.add(loan, m.name, m.telephone)
  RETURN report
```

Notice how the three procedures compose the ADTs: `Borrow` uses the catalogue (BST), the loan list (List) and the reservation map of queues (Map + Queue) in a single operation. This composition of abstract data types is the central skill of this chapter.

Task 5 — Class diagram.



Class diagram of the library system: each functional area is a class built on one ADT.

Task 6 — Trace of the demonstration scenario.

Day	Event	Queue for 978-001 (front → rear)	LoanList (978-001 only)
1	M01 borrows	empty	M01 (day 1)
1	M02 borrows → reserves	M02	M01 (day 1)
1	M03 reserves	M02, M03	M01 (day 1)
20	M01 returns; copy offered	M03	M02 (day 20)

On day 20, before the return is recorded, `OverdueReport(20)` scans the loan list: 20 - 1 = 19 > 14, so the loan of M01 is reported overdue, with the message "Ngong Blessing, 677 00 11 22 — Advanced Level Physics Revision, 5 days late" (due on day 1 + 14 = 15, so 20 - 15 = 5 days late). After the return, M02 holds the copy and M03 is now at the front of the queue — no quarrel, because the queue enforced the arrival order.

DID YOU KNOW? **Why "first come, first served" matters beyond libraries**

The same FIFO discipline designed here for Mrs. Eposi is used by operating systems to schedule print jobs, by call centres to answer callers in arrival order, and by mobile money platforms to process transactions fairly. Choosing the right ADT is not an academic exercise: it is a design decision with visible social consequences — here, the end of quarrels in front of the loans desk.

KEY POINTS **What this activity demonstrates**

A real system is built by composing several ADTs: a **BST** for fast ordered access to the catalogue, **maps** for key-based registers, a **list** for the loans, and **queues** for fair reservations. The design work — choosing the ADT, writing the interface with preconditions and

postconditions, and tracing the operations — is exactly what the GCE Advanced Level expects before any line of real code is written.

EXERCISE 6 **Extension: a stricter rule**

The committee adds a new rule: a member who already has an overdue loan may not borrow or reserve any other book. Modify the pseudocode of `Borrow` to enforce this rule, stating clearly which ADT operation you add or reuse, and write the precondition that must now appear in the `Borrow` specification.

SOLUTION **Extension: a stricter rule**

Add a function `LoanList.hasOverdue(memberNo, today) → Boolean` that scans the loan list and returns true if some current loan of that member satisfies *today* − *dateBorrowed* > 14. In `Borrow`, immediately after the 3-book check, insert:

```
IF LoanList.hasOverdue(memberNo, today) THEN
  output "Settle your overdue loan first"; STOP
```

The new precondition of `Borrow` is: the member has fewer than 3 current loans and no overdue loan. No new ADT is needed: the existing List simply gains one more scanning operation, which confirms that the List choice remains appropriate.

6 METHODS AND WORKED EXAMPLES

7 ABSTRACT DATA TYPES: FOUNDATIONS

7.1 Method 1: Specifying an Abstract Data Type

METHOD **Writing a complete ADT specification**

To specify an **abstract data type** rigorously, as expected at the GCE Advanced Level, proceed in numbered steps:

1. Name and purpose. Give the ADT a name (e.g. `Stack`, `Queue`, `List`) and state in one sentence what collection of values it models.
2. Domain of values. Describe the set of possible values abstractly (e.g. "a finite sequence of integers"), without mentioning arrays, pointers or any implementation detail.
3. Operations (the interface). For each operation give: its name, its inputs (parameters with types), its output (return type), and its effect. Distinguish constructors (create the ADT), accessors/observers (inspect without modifying, e.g. `isEmpty`, `top`) and mutators/transformers (modify the state, e.g. `push`, `pop`).
4. Preconditions. State when an operation is allowed (e.g. `pop` requires a non-empty stack).
5. Axioms / behaviour rules. State the laws linking the operations, e.g. `pop(push(S, x)) = S` and `top(push(S, x)) = x`. These axioms define the ADT independently of any implementation.

Reflex: if your specification mentions "array index" or "pointer", you have slipped into implementation — remove it. The golden rule is what the operations do, never how.

WORKED EXAMPLE 6 **Specifying the ADT Stack**

A school in Bamenda keeps a pile of marked scripts: the last script placed on the pile is the first one taken away. Give a complete abstract specification of the ADT `Stack` modelling this pile, including constructors, accessors, mutators, preconditions and two axioms.

SOLUTION

Solution. 1. Name and purpose. `Stack`: models a collection of items managed in **LIFO** or-

der (Last In, First Out).
2. Domain of values. A finite sequence of items of type `Item`, with a distinguished end called the top.
3. Operations.

Operation	Signature	Effect
`create()`	→ `Stack`	Constructor: returns a new empty stack.
`isEmpty(S)`	`Stack` → `Boolean`	Accessor: true iff S contains no item.
`push(S, x)`	`Stack` × `Item` → `Stack`	Mutator: places x on the top of S.
`pop(S)`	`Stack` → `Stack`	Mutator: removes the top item. Precondition: S not empty.
`top(S)`	`Stack` → `Item`	Accessor: returns the top item without removing it. Precondition: S not empty.

4. Axioms. For every stack S and item x:

$$\texttt{pop}(\texttt{push}(S, x)) = S$$
$$\texttt{top}(\texttt{push}(S, x)) = x$$
$$\texttt{isEmpty}(\texttt{create}()) = \text{true}$$

Conclusion. Nothing in this specification says whether the stack is stored in an array or a linked list: that is precisely the point of abstraction. The LIFO behaviour is fully captured by the axioms.

7.2 Method 2: Simulating Stack and Queue Operations

METHOD **Tracing a sequence of push/pop or enqueue/dequeue**

1. Draw the structure as a column (stack, top at the top) or as a horizontal line (queue, front on the left, rear on the right).
2. Process the operations one at a time, from left to right, redrawing or updating the structure after each one.
3. For a **stack**: `push` adds on top; `pop` removes from the same end (LIFO).
4. For a **queue**: `enqueue` adds at the rear; `dequeue` removes from the opposite end (FIFO).
5. Before any `pop`/`dequeue`/`top`, check the precondition (structure non-empty); if violated, signal underflow.
6. If asked for output, record the value returned by each `pop`/`dequeue` in order.

Reflex: a stack reverses order; a queue preserves order. Use this to check your final answer quickly.

WORKED EXAMPLE 7 **Stack and queue trace**

Consider the sequence of operations, applied in order to an initially empty stack S and an initially empty queue Q: `push(S,5); push(S,8); push(S,3); pop(S); enqueue(Q,7); enqueue(Q, pop(S)); enqueue(Q,2); dequeue(Q); push(S, dequeue(Q))`. Give the final content of S (top first) and of Q (front first), and the list of all values output by `pop`/`dequeue`.

SOLUTION

Solution. We trace step by step. Stack shown top-first, queue front-first.

Step	Operation	Stack S	Queue Q	Output
1	`push(S,5)`	5	—	—
2	`push(S,8)`	8, 5	—	—
3	`push(S,3)`	3, 8, 5	—	—
4	`pop(S)`	8, 5	—	3
5	`enqueue(Q,7)`	8, 5	7	—
6	`enqueue(Q, pop(S))`	5	7, 8	8
7	`enqueue(Q,2)`	5	7, 8, 2	—
8	`dequeue(Q)`	5	8, 2	7
9	`push(S, dequeue(Q))`	8, 5	2	8

Details of the tricky steps:

- Step 6: `pop(S)` returns 8 (the current top) and this value is enqueued; S becomes [5], Q becomes [7, 8].
- Step 9: `dequeue(Q)` removes the front, which is 8 (not 2: FIFO!), and pushes it onto S.

Final answer: S = [8, 5] (top first), Q = [2] (front first), and the output sequence is 3, 8, 7, 8. Check: the queue preserved order (7 entered before 8, and 7 left before 8); the stack reversed order (3 pushed last among {5, 8, 3} left first). Consistent.

7.3 Method 3: Evaluating Expressions with a Stack (Postfix)

METHOD **Evaluating a postfix (Reverse Polish) expression**

1. Scan the expression from left to right, one token at a time.
2. If the token is an operand (number), `push` it onto the stack.
3. If the token is an operator, `pop` the top two values: the first popped is the right operand, the second popped is the left operand. Compute `left op right` and `push` the result.
4. At the end, the stack must contain exactly one value: the answer. If it contains more, or if a pop fails, the expression is invalid.

WARNING **Order of operands**

For − and ÷, the order matters: in `6 2 −`, the result is 6 − 2 = 4, not 2 − 6. Always pop the right operand first.

WORKED EXAMPLE 8 **Postfix evaluation**

A calculator uses postfix notation. Evaluate the postfix expression

`5 1 2 + 4 * + 3 −`

showing the state of the stack after each token. Then give the equivalent infix expression.

SOLUTION

Solution. Stack shown with the top on the right.

Token	Stack after	Action
5	5	push 5
1	5, 1	push 1
2	5, 1, 2	push 2
+	5, 3	pop 2 (right), pop 1 (left); 1 + 2 = 3; push 3
4	5, 3, 4	push 4
*	5, 12	pop 4, pop 3; 3 × 4 = 12; push 12
+	17	pop 12, pop 5; 5 + 12 = 17; push 17
3	17, 3	push 3
−	14	pop 3 (right), pop 17 (left); 17 − 3 = 14; push 14

One value remains: the result is $\boxed{14}$.
Infix equivalent: reading the operations in the order they were applied: 5 + (1 + 2) × 4 − 3 =

5 + 12 − 3 = 14. ✓
Remark: this is exactly how compilers evaluate arithmetic expressions, and why the stack is a fundamental ADT in computer science.

8 BINARY TREE CONCEPTS AND REPRESENTATION

8.1 Method 4: Performing Tree Traversals

METHOD **Preorder, Inorder, Postorder on a binary tree**

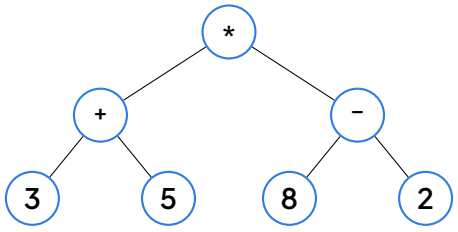
All three traversals are **recursive**. Let N = visit the node, L = traverse the left subtree, R = traverse the right subtree.

1. Preorder (NLR): visit the root first, then left, then right. Use: copying a tree, prefix expressions.
2. Inorder (LNR): left, then visit the root, then right. Use: on a binary search tree, gives values in ascending order.
3. Postorder (LRN): left, right, then the root last. Use: deleting a tree, postfix expressions.

Practical reflex for the exam: apply the recursion systematically — for each node, write the pattern (e.g. NLR) with the subtrees as placeholders, then expand each placeholder the same way. Never "guess" the order globally.

WORKED EXAMPLE 9 **Three traversals of an expression tree**

Consider the binary tree below, which represents an arithmetic expression:



Expression tree for (3 + 5) × (8 − 2).

Give the preorder, inorder and postorder traversals of this tree. Evaluate the expression

it represents.

SOLUTION

Solution. The root is $*$; its left subtree is rooted at $+$, its right subtree at $-$.
Preorder (NLR):

- Visit $*$; then preorder of left subtree: visit $+$, then 3, then 5; then preorder of right subtree: visit $-$, then 8, then 2.
- Result: $*\ +\ 3\ 5\ -\ 8\ 2$ (this is the prefix form of the expression).

Inorder (LNR):

- Left subtree: 3, $+$, 5; then root $*$; then right subtree: 8, $-$, 2.
- Result: $3 + 5 * 8 - 2$ (the infix form, up to parentheses).

Postorder (LRN):

- Left subtree: 3, 5, $+$; right subtree: 8, 2, $-$; then root $*$.
- Result: $3\ 5\ +\ 8\ 2\ -\ *$ (the postfix form).

Evaluation: the tree means $(3 + 5) \times (8 - 2) = 8 \times 6 = 48$.
Check with Method 3 on the postfix form $3\ 5\ +\ 8\ 2\ -\ *$: push 3, 5; $+$ gives 8; push 8, 2; $-$ gives $8 - 2 = 6$; $*$ gives $8 \times 6 = 48$. ✓

WARNING **Trap**

In inorder, $3 + 5 * 8 - 2$ looks ambiguous because of operator precedence; the tree structure itself removes all ambiguity. In an exam, add parentheses when interpreting an expression tree: $(3 + 5) * (8 - 2)$.

8.2 Method 5: Building and Searching a Binary Search Tree

METHOD **Inserting a sequence into a BST**

A **binary search tree** (BST) satisfies: for every node, all values in its left subtree are smaller, all values in its right subtree are larger (assuming distinct keys).

1. The first value becomes the root.
2. For each new value, start at the root and compare: smaller → go left; larger → go right.
3. Repeat until you reach an empty position; insert the new node there as a leaf.
4. To search: same comparisons; stop when the value is found (success) or an empty position is reached (failure). The number of comparisons equals the depth reached.

5. To list values in ascending order: perform an inorder traversal.

Reflex: the shape of the BST depends entirely on the order of insertion; a sorted input produces a degenerate (linear) tree — the worst case.

WORKED EXAMPLE 10 **Constructing a BST and analysing searches**

The results (out of 20) of eight candidates in a Computer Science test in Buea are recorded in the order they were marked:

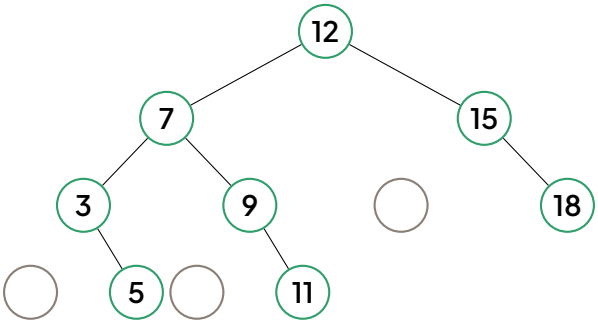
12, 7, 15, 3, 9, 18, 11, 5

1. Draw the BST obtained by inserting these values in the given order.
2. Give the inorder traversal and comment.
3. Count the comparisons needed to search for 11 and to search for 14.

SOLUTION

Solution. 1. Construction, step by step:

- 12 → root.
- 7 < 12 → left child of 12.
- 15 > 12 → right child of 12.
- 3 < 12, 3 < 7 → left child of 7.
- 9 < 12, 9 > 7 → right child of 7.
- 18 > 12, 18 > 15 → right child of 15.
- 11 < 12, 11 > 7, 11 > 9 → right child of 9.
- 5 < 12, 5 < 7, 5 > 3 → right child of 3.



BST built from the insertion order 12, 7, 15, 3, 9, 18, 11, 5.

2. Inorder traversal (LNR): left subtree of 12 gives 3, 5, 7, 9, 11; then 12; then right subtree gives 15, 18.

$$3,\ 5,\ 7,\ 9,\ 11,\ 12,\ 15,\ 18$$

Comment: as expected, the inorder traversal of a BST lists the marks in ascending order — this is the principle of the tree sort.

3. Search comparisons:

- Search 11: compare with 12 (go left, 1), with 7 (go right, 2), with 9 (go right, 3), with 11 (found, 4). Total: 4 comparisons.
- Search 14: compare with 12 (right, 1), with 15 (left, 2); the left child of 15 is empty ⇒ failure after 2 comparisons, and we know 14 would be inserted as the left child of 15.

Remark: the tree has height 4 (levels 1 to 4), so any search needs at most 4 comparisons here, instead of up to 8 in an unsorted list.

8.3 Method 6: Representing a Binary Tree (Array and Linked)

METHOD **Choosing and using a representation of a binary tree**

1. Array (sequential) representation: store the node at index i; its left child is at index $2i$, its right child at $2i + 1$ (root at index 1). Empty positions are marked with a sentinel (e.g. -1 or `null`).
2. Linked representation: each node is a record with three fields: `data`, `left` (pointer to left child), `right` (pointer to right child); empty children hold `null`.
3. Choosing: the array form is compact and simple for complete/nearly complete trees (e.g. heaps) but wastes memory for sparse or skewed trees; the linked form uses exactly one node per value and makes insertion/deletion easy, at the cost of pointer storage.

Reflex for the exam: to convert a drawn tree into an array, number the nodes level by level, left to right (breadth-first order), leaving gaps for missing children.

WORKED EXAMPLE 11 **Array representation of a given tree**

The BST of the previous exercise contains the values {12, 7, 15, 3, 9, 18, 11, 5}.

1. Give the array representation of this tree using the rule "children of node i at $2i$ and $2i + 1$", with 0 marking empty cells.
2. How many cells does the array need? Comment on memory usage.

3. State one advantage of the linked representation for this particular tree.

SOLUTION

Solution. 1. Place the root 12 at index 1. Its children 7 and 15 go to indices 2 and 3. Children of 7 (index 2): 3 at 4, 9 at 5. Children of 15 (index 3): empty at 6, 18 at 7. Children of 3 (index 4): empty at 8, 5 at 9. Children of 9 (index 5): empty at 10, 11 at 11.

Index	1	2	3	4	5	6	7	8	9	10	11
Value	12	7	15	3	9	0	18	0	5	0	11

Check with the rule: node 5 is at index 9; its parent is at index $\lfloor 9/2 \rfloor = 4$, which holds 3 — correct, since 5 is the right child of 3.
2. The deepest node (5 or 11) sits at index 9 and 11, so the array needs 11 cells for only 8 values: 3 cells are wasted (indices 6, 8, 10). For a more unbalanced tree the waste grows exponentially with height.
3. Advantage of the linked representation: it allocates exactly one node per value (8 nodes), with no empty slots, and inserting a new mark (e.g. 14) only requires creating one node and updating one pointer — no need to resize or shift an array.

9 APPLICATIONS OF ADTS

9.1 Method 7: Writing Recursive Algorithms on ADTs

METHOD **Designing a recursive algorithm on a binary tree**

1. Base case: decide what happens for an empty tree (return 0, `true`, nothing, …). This case must be written first and must stop the recursion.
2. Recursive case: express the answer for a tree in terms of the answers for its left and right subtrees, combined with the root.
3. Check termination: each recursive call works on a strictly smaller tree, so the empty tree is always reached.
4. Trace the algorithm on a small example (trace table or call tree) before finalising.

Classic patterns: size = 1 + size(L) + size(R); height = 1 + max(height(L), height(R)); a traversal = visit root combined with two recursive calls in the appropriate order.

WORKED EXAMPLE 12 **Counting leaves and computing height — integration**

A school administration in Yaoundé stores a family tree of sponsored students as a binary tree. Each node is a record with fields `data`, `left`, `right`; an empty tree is `null`.

1. Write in pseudocode a function `countLeaves(T)` returning the number of leaves (nodes with no child) of tree T.
2. Write a function `height(T)` returning the height of T (an empty tree has height 0).
3. Trace `countLeaves` on the tree with root A, left child B (a leaf) and right child C whose left child is the leaf D.

SOLUTION

Solution. 1. Counting leaves. Base cases: empty tree → 0; a node with two empty children → 1. Otherwise, sum the leaves of both subtrees.

```
FUNCTION countLeaves(T) : Integer
  IF T = null THEN
    RETURN 0
  ELSE IF T.left = null AND T.right = null THEN
    RETURN 1
  ELSE
    RETURN countLeaves(T.left) + countLeaves(T.right)
  END IF
END FUNCTION
```

2. Height.

```
FUNCTION height(T) : Integer
  IF T = null THEN
    RETURN 0
  ELSE
    RETURN 1 + max(height(T.left), height(T.right))
  END IF
END FUNCTION
```

3. Trace of `countLeaves` on the described tree (A has children B and C; C has left child D; B and D are leaves):

Call	Reason	Returns
`countLeaves(A)`	*A* has children → sum of subtrees	1 + 1 = 2
`countLeaves(B)`	*B* is a leaf	1
`countLeaves(C)`	*C* has a left child → sum	1 + 0 = 1
`countLeaves(D)`	*D* is a leaf	1
`countLeaves(null)`	right child of *C* is empty	0

The function returns $\boxed{2}$, which is correct: the leaves are *B* and *D*.
Termination check: every recursive call is made on a strictly smaller subtree, and the empty tree is a base case, so the recursion always ends. The height of this tree is 1 + max(1, 2) = 3, also verified by the algorithm.

10 INTEGRATION ACTIVITY & EXAM PREPARATION

KEY POINTS **Summary of exam reflexes**

- An ADT is specified by its operations and axioms, never by its implementation.
- Stack = LIFO (reverses order); Queue = FIFO (preserves order). Always check the non-empty precondition before `pop`/`dequeue`.
- Postfix evaluation: operand → push; operator → pop two (right first!), compute, push.
- Traversals: preorder NLR, inorder LNR, postorder LRN. Inorder of a BST = ascending order.
- BST insertion: smaller left, larger right; the shape depends on the insertion order.
- Array representation: children of i at $2i$ and $2i + 1$; linked representation: `data`, `left`, `right`.
- Every recursive tree algorithm needs an explicit base case (empty tree) and a recursive combination.

11 EXERCISES

11.1 I check my knowledge

EXERCISE 7 **MCQ: choosing the right ADT**

For each of the following situations, choose the most appropriate abstract data type (A, B, C or D) and justify your choice in one sentence.

1. A printer server processes print jobs in the order they arrive.
   A. Stack B. Queue C. Binary search tree D. Array
2. A text editor must support an Undo command that reverses the most recent action first.
   A. Stack B. Queue C. Linked list D. Hash table
3. A dictionary must allow very fast search, insertion and deletion of words kept in alphabetical order.
   A. Stack B. Queue C. Binary search tree D. Simple array
4. A program must evaluate an arithmetic expression written in postfix notation.
   A. Stack B. Queue C. Graph D. Deque

EXERCISE 8 **True or false — justify**

State whether each statement is true or false, and justify your answer in one or two sentences.

1. In a stack, the element removed is always the one that has been in the structure the longest.
2. Every node of a binary tree has at most two children.
3. A complete binary tree of height h contains exactly $2^{h+1} - 1$ nodes.
4. An ADT specifies what operations do, independently of how they are implemented.
5. The in-order traversal of a binary search tree always visits the keys in ascending order.

EXERCISE 9 **Key definitions**

Give a precise definition of each of the following terms, as expected at the GCE Advanced Level:

1. Abstract Data Type (ADT);
2. FIFO structure and LIFO structure (give one example of each);
3. the height of a binary tree and the depth of a node;
4. a full binary tree and a complete binary tree;
5. a binary search tree (BST).

EXERCISE 10 **Direct calculations on binary trees**

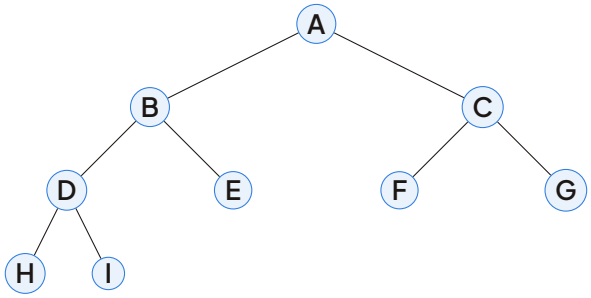
A complete binary tree has height $h = 4$ (the root is at depth 0).

1. How many nodes does the tree contain?
2. How many leaves does it contain?
3. If this tree is stored in an array using the standard indexing where the root is at index 1, at which indices are the left child, the right child and the parent of the node stored at index $i = 6$?
4. What is the minimum number of nodes a binary tree of height 4 can contain? Draw such a tree.

11.2 I practise

EXERCISE 11 **Tree traversals**

Consider the following binary tree, where the root contains the value *A*:



1. Give the sequence of nodes visited by the pre-order traversal.
2. Give the sequence of nodes visited by the in-order traversal.
3. Give the sequence of nodes visited by the post-order traversal.

4. Give the sequence of nodes visited by the level-order (breadth-first) traversal, and state which ADT is used to implement it.

EXERCISE 12 **Infix to postfix and evaluation**

Consider the infix expression $(A + B) * (C - D / E) + F$.

1. Using a stack, convert this expression into postfix notation. Show the content of the stack and the output produced after reading each token, in a trace table.
2. Evaluate the postfix expression obtained for $A = 2$, $B = 3$, $C = 10$, $D = 8$, $E = 4$, $F = 1$, showing the state of the evaluation stack at each step.

EXERCISE 13 **Building a binary search tree**

1. Write, in clear pseudocode, a recursive algorithm `Insert(root, key)` that inserts a key into a binary search tree.
2. Starting from an empty tree, draw the binary search tree obtained after inserting, in this order, the keys: 50, 30, 70, 20, 40, 60, 80.
3. What is the height of the tree obtained? Give the in-order traversal of this tree and comment on the result.
4. State the time complexity of the insertion in the best case and in the worst case, for a BST with n nodes.

EXERCISE 14 **A queue built from two stacks**

A programmer has only the ADT Stack (with operations `Push`, `Pop`, `IsEmpty`) available, and must implement the ADT Queue (operations `Enqueue`, `Dequeue`) using two stacks S_1 and S_2.

1. Write pseudocode for `Enqueue(x)` and `Dequeue()` using only the stack operations.
2. Trace your algorithms on the following sequence of operations, showing the content of S_1 and S_2 after each operation: `Enqueue(5)`, `Enqueue(8)`, `Enqueue(3)`, `Dequeue()`, `Enqueue(7)`, `Dequeue()`.
3. Explain why the amortised cost of `Dequeue` is $O(1)$, even though a single `Dequeue` may take $O(n)$ in the worst case.

11.3 I prepare for the GCE

EXERCISE 15 **Exam-style: choosing and justifying an ADT (10 marks)**

The students' association of a large secondary school in Yaoundé is developing a mobile application to manage a music playlist for the end-of-year party. The requirements are:

- songs can be added quickly at the beginning and at the end of the playlist;
- the DJ frequently removes the song at the beginning once it has been played;
- occasionally, a "priority request" song must be inserted at the beginning;
- the total number of songs is not known in advance.

1. Define the term abstract data type and explain why it is useful to reason in terms of ADTs before choosing an implementation. (3 marks)
2. Among the following ADTs — array, stack, simple queue, double-ended queue (deque), singly linked list — select the one you consider most suitable for this playlist. Justify your choice by analysing the time complexity of each required operation for at least two candidate structures, presented in a comparison table. (5 marks)
3. Write pseudocode for the operation that inserts a song at the beginning of your chosen structure. (2 marks)

EXERCISE 16 **Exam-style: binary tree representations and traversals (12 marks)**

A school keeps, for each class, a binary tree recording the results of a quiz competition: each internal node contains the name of a match, and each leaf contains the name of a candidate.

1. Distinguish clearly between a binary tree and a binary search tree. (2 marks)
2. Describe the array-based (sequential) representation of a binary tree, giving the formulas that link the index of a node to the indices of its children and its parent. (3 marks)
3. Describe the linked representation of a binary tree, specifying the fields of a node. (2 marks)
4. Compare the two representations in terms of memory usage (consider the case of a very unbalanced tree), ease of insertion of new nodes, and speed of access to the children of a node. Present your comparison in a table. (3 marks)
5. A binary search tree contains the keys 40, 25, 60, 15, 35, 50, 70 inserted in that order. Draw the tree and give its pre-order and post-order traversals. (2 marks)

EXERCISE 17 **Exam-style: expression evaluation and integration of ADTs (14 marks)**

A calculator application evaluates arithmetic expressions typed by the user in ordinary (infix) notation.

1. Explain why a computer cannot evaluate an infix expression directly in one left-to-right pass, and why postfix (Reverse Polish) notation is easier to evaluate. (3 marks)
2. Convert the infix expression $(A + B) * (C - D / E) + F$ to postfix, showing the stack trace. (4 marks)
3. Write, in pseudocode, the algorithm that evaluates a postfix expression using a stack. (3 marks)
4. Trace your algorithm on the postfix expression obtained in question 2 with $A = 2$, $B = 3$, $C = 10$, $D = 8$, $E = 4$, $F = 1$. (2 marks)
5. The application must also keep a history of the last 20 evaluated expressions so that the user can revisit the most recent one first. Which ADT do you recommend for this history? Justify your answer. (2 marks)

12 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ: choosing the right ADT**

1. B. Queue. A printer server must process jobs in the order they arrive (First-In-First-Out), which is the defining behaviour of a queue.
2. A. Stack. The Undo command must reverse the most recent action first (Last-In-First-Out), exactly the behaviour of a stack.
3. C. Binary search tree. A dictionary requires fast search, insertion and deletion while maintaining alphabetical order; a balanced BST provides $O(\log n)$ for all three operations.
4. A. Stack. Evaluating a postfix expression requires storing operands and intermediate results and retrieving the two most recent operands when an operator is read, which is a classic stack application.

SOLUTION **Exercise 2 — True or false — justify**

1. False. In a stack the element removed is the one most recently inserted (LIFO — Last-In-First-Out). The element that has been in the structure the longest is removed in a queue (FIFO).
2. True. By definition, a binary tree is a tree in which every node has at most two children (a left child and/or a right child).
3. False. A perfect (or full) binary tree of height h contains exactly $2^{h+1} - 1$ nodes. A complete binary tree of height h contains between 2^h and $2^{h+1} - 1$ nodes.
4. True. An Abstract Data Type specifies the logical behaviour (what operations do) without specifying the implementation details (how they are done).
5. True. The in-order traversal (Left-Root-Right) of a binary search tree visits keys in non-decreasing order because all keys in the left subtree are smaller than the root and all keys in the right subtree are larger.

SOLUTION **Exercise 3 — Key definitions**

1. Abstract Data Type (ADT): A mathematical model that defines a data structure purely

by its behaviour: a set of values and a set of operations on those values, without specifying how the values are stored or how the operations are implemented in memory.

2. FIFO structure (First-In-First-Out): A structure where the first element inserted is the first one removed. Example: Queue. LIFO structure (Last-In-First-Out): A structure where the last element inserted is the first one removed. Example: Stack.
3. Height of a binary tree: The number of edges on the longest path from the root to a leaf (or the maximum depth of any node). Depth of a node: The number of edges on the path from the root to that node (the root has depth 0).
4. Full binary tree: A binary tree in which every node has either 0 or 2 children (no node has exactly one child). Complete binary tree: A binary tree in which all levels are completely filled except possibly the last level, which is filled from left to right without gaps.
5. Binary Search Tree (BST): A binary tree where for every node, all keys in its left subtree are strictly less than the node's key, and all keys in its right subtree are strictly greater than the node's key. This property holds recursively for all subtrees.

SOLUTION **Exercise 4 — Direct calculations on binary trees**

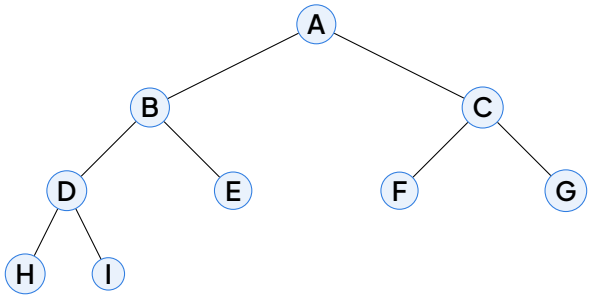
1. A complete binary tree of height $h = 4$ (root depth 0) that is also perfect (all levels full) contains $2^{h+1} - 1 = 2^5 - 1 = 31$ nodes.
2. The number of leaves in a perfect binary tree of height h is $2^h = 2^4 = 16$.
3. Standard 1-based array indexing:
   - Left child of node i: $2i = 2 \times 6 = 12$.
   - Right child of node i: $2i + 1 = 13$.
   - Parent of node i: $\lfloor i/2 \rfloor = \lfloor 6/2 \rfloor = 3$.
4. The minimum number of nodes in a binary tree of height 4 is $h + 1 = 5$ (a completely skewed tree, each node having only one child).



Minimum-node binary tree of height 4 (skewed).

SOLUTION **Exercise 5 — Tree traversals**

The tree:



Binary tree for traversal exercises.

1. Pre-order (Root–Left–Right): *A B D H I E C F G*
2. In-order (Left–Root–Right): *H D I B E A F C G*
3. Post-order (Left–Right–Root): *H I D E B F G C A*
4. Level-order (Breadth-first): *A B C D E F G H I*. The ADT used to implement level-order traversal is a Queue (FIFO).

SOLUTION **Exercise 6 — Infix to postfix and evaluation**

1. Infix to Postfix conversion trace: Expression: $(A + B) * (C - D / E) + F$

Token	Action	Stack (top at right)	Output
(	Push (	(	
A	Output	(	A
+	Push +	(+	A
B	Output	(+	A B
)	Pop until (	(empty)	A B +
*	Push *	*	A B +
(	Push (	* (	A B +
C	Output	* (	A B + C
-	Push -	* (-	A B + C
D	Output	* (-	A B + C D
/	Push / (higher prec than -)	* (- /	A B + C D
E	Output	* (- /	A B + C D E
)	Pop until (	*	A B + C D E / -
+	Pop * (higher prec), push +	+	A B + C D E / - *
F	Output	+	A B + C D E / - * F
End	Pop +	(empty)	A B + C D E / - * F +

Postfix expression: $A\ B\ +\ C\ D\ E\ /\ -\ *\ F\ +$

2. Evaluation of postfix for $A = 2, B = 3, C = 10, D = 8, E = 4, F = 1$:

Token	Action	Stack (top at right)
2	Push 2	2
3	Push 3	2, 3
+	Pop 3,2; push 2+3=5	5
10	Push 10	5, 10
8	Push 8	5, 10, 8
4	Push 4	5, 10, 8, 4
/	Pop 4,8; push 8/4=2	5, 10, 2
-	Pop 2,10; push 10−2=8	5, 8
*	Pop 8,5; push 5*8=40	40
1	Push 1	40, 1
+	Pop 1,40; push 40+1=41	41

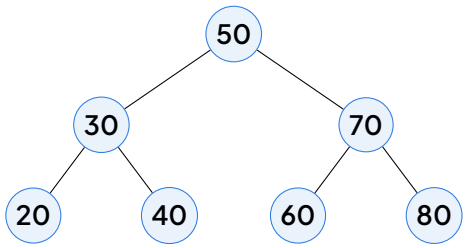
Final result: 41.

SOLUTION **Exercise 7 — Building a binary search tree**

1. Recursive Insert pseudocode:

```
(a) Function Insert(root, key)
(b)   If root = NULL Then
(c)     Create new node with key
(d)     Return new node
(e)   Else If key < root.key Then
(f)     root.left = Insert(root.left, key)
(g)   Else If key > root.key Then
(h)     root.right = Insert(root.right, key)
(i)   Return root
(j) End Function
```

2. BST after inserting 50, 30, 70, 20, 40, 60, 80:



BST after inserting 50, 30, 70, 20, 40, 60, 80.

3. Height: The longest path from root (50) to a leaf (20, 40, 60, 80) has 2 edges. Height = 2. In-order traversal: 20 30 40 50 60 70 80. Comment: The in-order traversal of a BST always yields the keys in ascending (sorted) order.

4. Time complexity of insertion in a BST with n nodes:

   - Best case: $O(\log n)$ (tree is balanced, height $\approx \log_2 n$).
   - Worst case: $O(n)$ (tree is completely skewed, height = $n - 1$).

SOLUTION **Exercise 8 — A queue built from two stacks**

1. Pseudocode: Enqueue(x):

(a) `Push(S1, x)`

Dequeue():

(a) `If IsEmpty(S2) Then`

(b) `  While Not IsEmpty(S1) Do`

(c) `    Push(S2, Pop(S1))`

(d) `If IsEmpty(S2) Then`

(e) `  Return ERROR (queue empty)`

(f) `Else`

(g) `  Return Pop(S2)`

2. Trace:

Operation	S1 (top at right)	S2 (top at right)
Initial	empty	empty
Enqueue(5)	5	empty
Enqueue(8)	5, 8	empty
Enqueue(3)	5, 8, 3	empty
Dequeue()	empty	3, 8, 5 → Pop 5 — 3, 8
Enqueue(7)	7	3, 8
Dequeue()	7	3, 8 → Pop 8 — 3

(Note: After first Dequeue, S1 is emptied into S2, so S2 becomes [3,8,5] with 5 on top. Pop returns 5. S2 left with [3,8] top=8. Enqueue(7) pushes to S1. Second Dequeue pops 8 from S2.)

3. Amortised analysis: Each element is pushed onto S1 once, popped from S1 once, pushed onto S2 once, and popped from S2 once — at most four stack operations per element. Over a sequence of n operations, the total time is $O(n)$, so the amortised cost per operation is $O(1)$. A single `Dequeue` may take $O(n)$ when S2 is empty and S1 contains n elements, but this expensive transfer happens only once per element.

SOLUTION **Exercise 9 — Exam-style: choosing and justifying an ADT (10 marks)**

1. Definition of ADT (3 marks): An Abstract Data Type is a mathematical specification of a data structure defined by its set of values and the operations that can be performed on them, independently of any concrete implementation. It describes what the structure does, not how it does it. Usefulness: Reasoning with ADTs allows the program-

mer to focus on the logical requirements of the problem (e.g., "I need to add/remove at both ends") without being distracted by implementation details (arrays vs pointers). It enables modular design: the implementation can be changed (e.g., from array to linked list) without affecting the rest of the program, and it facilitates correctness proofs and complexity analysis at the abstract level.

2. Choice and justification (5 marks): The most suitable ADT is the Double-Ended Queue (Deque). Comparison table:

Operation required	Deque	Singly Linked List (with tail pointer)	Array (dynamic)
Add at beginning	$O(1)$	$O(1)$	$O(n)$ (shift all elements)
Add at end	$O(1)$	$O(1)$	$O(1)$ amortised
Remove from beginning	$O(1)$	$O(1)$	$O(n)$ (shift all elements)
Insert priority at beginning	$O(1)$	$O(1)$	$O(n)$
Unknown size	Dynamic	Dynamic	Requires resizing (amortised $O(1)$)

Justification: A Deque supports all four required operations in $O(1)$ worst-case time (if implemented with a doubly linked list or circular buffer). A simple Queue or Stack only allows operations at one end. An Array is inefficient for insertions/removals at the beginning. A Singly Linked List with a tail pointer also achieves $O(1)$ for all operations, but a Deque is the standard ADT that directly models "add/remove at both ends" and is the most precise abstraction for the requirements.

3. Pseudocode for insert at beginning (2 marks):

```
(a) Function AddFront(song)
(b)   Create new node N with song
(c)   If IsEmpty() Then
(d)     front = rear = N
(e)   Else
(f)     N.next = front
(g)     front.prev = N    (for doubly linked list)
(h)     front = N
(i) End Function
```

(If using a circular array implementation, decrement front index modulo capacity and store song.)

SOLUTION **Exercise 10 — Exam-style: binary tree representations and traversals (12 marks)**

1. Binary tree vs BST (2 marks): A binary tree is a hierarchical structure where each node has at most two children (left and right), with no ordering constraint on the values. A binary search tree (BST) is a binary tree that satisfies the BST property: for every node, all keys in its left subtree are strictly less than the node's key, and all keys in its right subtree are strictly greater. This property enables efficient search ($O(h)$).

2. Array-based representation (3 marks): Nodes are stored in an array starting at index 1 (root). For a node at index i:

   - Left child at index $2i$ (if $2i \le n$).
   - Right child at index $2i + 1$ (if $2i + 1 \le n$).
   - Parent at index $\lfloor i / 2 \rfloor$ (if $i > 1$).

   Empty positions (missing nodes) must be represented by a sentinel value (e.g., -1 or a special marker), which wastes space for unbalanced trees.

3. Linked representation (2 marks): Each node is a record (struct/class) containing three fields:

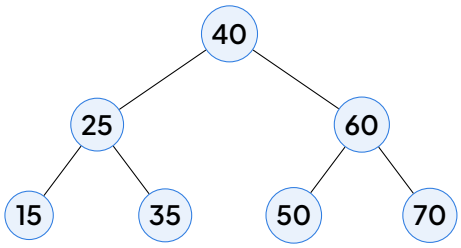
   - `data`: the value stored.
   - `left`: pointer/reference to left child (or NULL).
   - `right`: pointer/reference to right child (or NULL).

   The tree is accessed via a pointer to the root node.

4. Comparison table (3 marks):

Criterion	Array-based	Linked
Memory usage (unbalanced tree)	Poor: allocates space for all potential nodes up to max index; many gaps waste memory.	Good: allocates exactly one record per actual node; no gaps.
Ease of insertion	Difficult: inserting a node may require shifting many elements or resizing array; gaps complicate structure.	Easy: create new node, adjust parent's pointer; no shifting needed.
Speed of access to children	Very fast: direct index calculation $2i$, $2i + 1$ (O(1) arithmetic).	Fast: pointer dereference (O(1) but involves memory indirection).

5. BST construction and traversals (2 marks): Insert 40, 25, 60, 15, 35, 50, 70.



BST after inserting 40, 25, 60, 15, 35, 50, 70.

Pre-order: 40 25 15 35 60 50 70 Post-order: 15 35 25 50 70 60 40

SOLUTION **Exercise 11 — Exam-style: expression evaluation and integration of ADTs (14 marks)**

1. Why infix is hard, postfix is easy (3 marks): Infix notation requires the evaluator to handle operator precedence (e.g., $*$ before $+$) and parentheses, which means the next operation to perform is not always the next operator read; one must look ahead or use a stack to delay operations. Postfix (Reverse Polish) notation places each operator after its operands, so the expression can be evaluated in a single left-to-right pass using a stack: when an operand is read, push it; when an operator is read, pop the required operands, apply the operator, push the result. No precedence rules or paren-

theses are needed.

2. Infix to Postfix conversion (4 marks): (Same expression as Exercise 6) $(A+B)*(C-D/E)+F$
Trace table:

Token	Action	Stack	Output
(	Push	(	
A	Output	(	A
+	Push	(+	A
B	Output	(+	A B
)	Pop to (	empty	A B +
*	Push	*	A B +
(	Push	* (	A B +
C	Output	* (	A B + C
-	Push	* (-	A B + C
D	Output	* (-	A B + C D
/	Push	* (- /	A B + C D
E	Output	* (- /	A B + C D E
)	Pop to (	*	A B + C D E / -
+	Pop *, push +	+	A B + C D E / - *
F	Output	+	A B + C D E / - * F
End	Pop +	empty	A B + C D E / - * F +

Postfix: $A\ B\ +\ C\ D\ E\ /\ -\ *\ F\ +$

3. Postfix evaluation algorithm (3 marks):

(a) `Function EvaluatePostfix(expression)`

(b) `  Create empty stack S`

(c) `  For each token in expression Do`

(d) `    If token is an operand Then`

(e) `      Push(S, token)`

(f) `    Else (token is an operator)`

(g) `      op2 = Pop(S)`

(h) `      op1 = Pop(S)`

(i) `      result = op1 token op2`

(j) `      Push(S, result)`

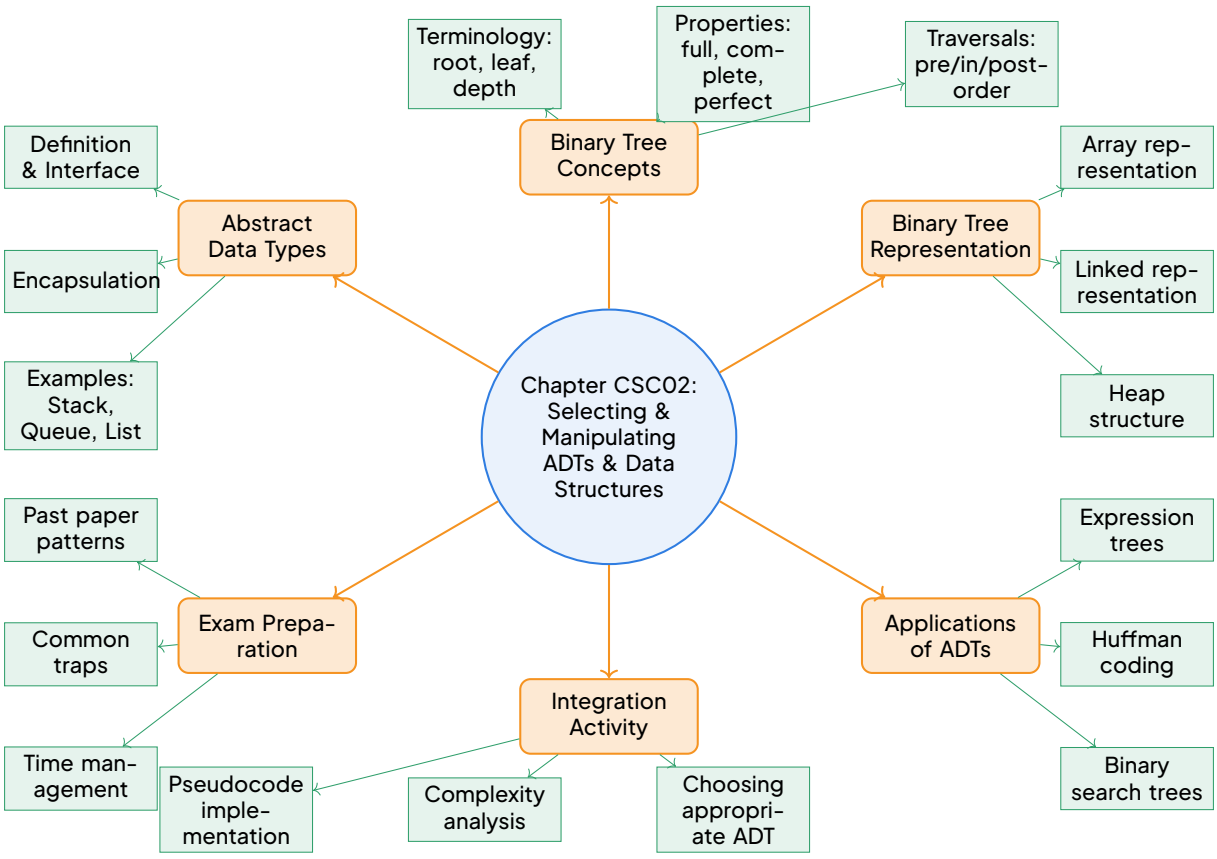
(k) `Return Pop(S)` (final result)

(l) `End Function`

4. Trace with values (2 marks): (Same as Exercise 6) $A = 2, B = 3, C = 10, D = 8, E = 4, F = 1$ Stack trace: 2 → 2,3 → + → 5 → 5,10 → 5,10,8 → 5,10,8,4 → / → 5,10,2 → - → 5,8 → * → 40 → 40,1 → + → 41. Final result: 41.

5. History ADT (2 marks): The Stack (LIFO) is the most appropriate ADT for the history. Justification: The requirement is to "revisit the most recent one first". A stack naturally provides $O(1)$ access to the most recent item (top) and $O(1)$ insertion (push) when a new expression is evaluated. Since the history is limited to 20 items, a bounded stack implemented as a circular buffer can discard the oldest (bottom) item when full, or simply ignore new entries if the specification says "last 20". A queue would give the oldest first, which is the opposite of the requirement.

13 SUMMARY SHEET



Mind map of Chapter CSC02: Selecting and manipulating ADTs and Data Structures

KEY POINTS **Key Definitions**

- **Abstract Data Type (ADT)**: a mathematical model that defines a set of values and a collection of operations on those values, without specifying how the values are stored or how the operations are implemented.
- **Encapsulation**: hiding the internal representation of an ADT; users can only access the data through the defined operations (the interface).
- **Binary Tree**: a hierarchical structure in which each node has at most two children, referred to as the left child and the right child.
- **Root**: the unique node with no parent. **Leaf**: a node with no children. **Height** of a tree: the number of edges on the longest path from the root to a leaf (a tree with only a root has height 0).

- **Full Binary Tree**: every node has either 0 or 2 children.
- **Complete Binary Tree**: all levels are completely filled except possibly the last, which is filled from left to right.
- **Perfect Binary Tree**: all internal nodes have two children and all leaves are at the same level.
- **Traversal**: a systematic way of visiting every node exactly once: pre-order (root, left, right), in-order (left, root, right), post-order (left, right, root).
- **Heap**: a complete binary tree satisfying the heap property: in a max-heap every parent is greater than or equal to its children; in a min-heap every parent is less than or equal to its children.
- **Binary Search Tree (BST)**: a binary tree in which, for every node, all keys in the left subtree are smaller than the node's key and all keys in the right subtree are greater.

KEY POINTS **Laws, Formulas and Theorems (examinable)**

- Maximum number of nodes in a binary tree of height h: $N_{max} = 2^{h+1} - 1$ (height of root = 0).
- Minimum number of nodes in a binary tree of height h: $N_{min} = h + 1$ (completely skewed tree).
- For a complete binary tree with n nodes, the height is $h = \lfloor \log_2 n \rfloor$.
- Array representation indices (1-based indexing): the left child of the node at index i is at index $2i$, the right child at index $2i + 1$, and the parent of the node at index i is at index $\lfloor i/2 \rfloor$.
- Time complexities for a BST (average case, balanced tree): search, insertion and deletion are $O(\log n)$; in the worst case (degenerate tree) they degrade to $O(n)$.
- Heap operations: insertion $O(\log n)$, extraction of the maximum/minimum $O(\log n)$, building a heap from n elements $O(n)$.
- Any traversal of a tree with n nodes visits each node exactly once, hence runs in $O(n)$ time and uses $O(h)$ auxiliary space (the recursion stack, where h is the height).

METHOD **Essential Step-by-Step Methods**

1. Choosing an ADT: list the operations the problem requires (push/pop, en-

queue/dequeue, search, insert, delete, priority access) and match them to the appropriate structure: Stack, Queue, List, Tree or Graph.

2. Designing a binary tree: define the node structure (data field, left pointer, right pointer); then choose the representation — array-based for complete trees, linked for dynamic or sparse trees.
3. Implementing traversals: write recursive pseudocode with a clear base case (empty tree); for an iterative version, use an explicit stack.
4. Heap operations: use `heapify` (sift-down) when building a heap; `siftUp` after inserting at the last position; `siftDown` after replacing the root during extraction.
5. BST operations: search recursively by comparing the key with the current node; insert at the leaf position found by an unsuccessful search; delete by treating three cases — leaf node, node with one child, node with two children (replace with the in-order successor, then delete that successor).
6. Complexity analysis: count the basic operations as a function of the input size n; express the result in Big-O notation; always distinguish best, average and worst cases.

WARNING **Common Mistakes**

- Confusing height (number of edges on the longest downward path) with level or depth counted in nodes; in the GCE convention the height of a tree containing only the root is 0.
- Forgetting that the array representation is only memory-efficient for complete trees; using it for sparse or skewed trees wastes a large amount of memory.
- Mixing up pre-order, in-order and post-order sequences; always draw a small tree and trace the traversal by hand before answering.
- Assuming BST operations are always $O(\log n)$; they degrade to $O(n)$ when the tree becomes unbalanced (for example after inserting already-sorted data).
- In heap `extractMax`, neglecting to call `heapify` (sift-down) on the new root after swapping it with the last element.
- Writing recursive algorithms without a base case, which leads to infinite recursion.
- Omitting the space complexity of recursive traversals: the recursion stack can grow as deep as the height h of the tree.
- Using 0-based array formulas ($2i+1, 2i+2$) when the question states 1-based indexing ($2i, 2i+1$) — read the question carefully.

KEY POINTS **GCE Examination Tips**

- Past papers frequently ask: "Draw the binary search tree resulting from inserting the sequence 50, 30, 70, 20, 40, 60, 80 into an initially empty BST." Practise drawing such trees step by step, showing each insertion.
- Expect a question on converting an infix expression to postfix form using a stack, and then evaluating the postfix expression with a stack.
- Be ready to write pseudocode for `inOrderTraversal(node)` and to state its time complexity $O(n)$ and space complexity $O(h)$.
- Know the array index formulas by heart; a typical question is: "A complete binary tree is stored in an array A[1..n]. Write down the index of the left child of the node stored at index i." (Answer: $2i$.)
- For the integration activity, you may be given a scenario (for example a school library system) and asked to select appropriate ADTs for cataloguing, the borrowing queue and indexing — justify each choice with the operations needed and their complexity.
- Manage your time: allow about 10 minutes for diagram drawing, 15 minutes for pseudocode, and 5 minutes for complexity justification.
- Use clear indentation and brief comments in pseudocode; examiners award marks for readability and structure.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+